

Leant2: Answers to the Expert Questions in Proposal 11

Mathematical foundations, concrete algorithms,
implementation boundaries, and an evaluation plan

Prepared for Vladimir Reshetnikov

21 September 2026

Abstract

The further-improvements proposal for Leant2 poses 31 expert questions in nine areas: search control, carrier invention, evaluation with holes, dependent elimination, general recursion, contracts, retrieval, ranking, and learned guidance. This article gives a separately identifiable answer to every question, grounded in a pinned repository snapshot, primary literature, and explicit mathematical arguments. Several apparent research barriers have useful bounded solutions: finite-state carrier inference becomes a finite constraint problem; type classes already have a dictionary-passing treatment in Hoogle+; and Lean exposes a concrete motive-construction API. Other ambitions need sharper contracts: finite observations do not determine a unique semantic carrier, open-goal counts are not generally admissible search bounds, and final kernel checking cannot repair an unsound pruning rule.

The proposed architecture keeps Lean’s native expressions and transactional state, adds dependency-indexed constraints and replayable proposal recipes, and separates fixed certification objectives from adaptive search priorities. It gives substitution-stable residual checking, capability-based recursion with a simulation obligation, local inductive contracts, and a retrieval abstraction that preserves dependencies conservatively. The recommendations include amendments to the acceptance boundary and result algebra, a staged implementation plan, and reproducible finite-model checks. No new Leant2 performance measurements or compiled Lean implementation are claimed.

Repository snapshot:

784664ff34e819422510a4d470ab07fe48bb736b

Target toolchain: `leanprover/lean4:v4.34.0`.

Scope: Proposal docs/proposals/11-further-improvements, with selected implementation and Lean API inspection.

Contents

1	Scope, evidence, and the principal decisions	3
1.1	What was investigated	3
1.2	How to read the claims	3
1.3	The existing evidence supports a targeted rewrite	3
2	The semantic contract before the algorithms	4
2.1	Six guarantees that should not share one label	4
2.2	What the inspected acceptance boundary does, and what to strengthen	5
2.3	A result algebra with enough information	6
2.4	Corrections that affect the research plan	6
3	R1: Search control and shared metavariables	7
3.1	R1.Q1. Search without duplicating all dependent work	7
3.2	R1.Q2. An admissible remaining-cost bound	8
3.3	R1.Q3. Learning search costs without changing the promise	8
3.4	R1.Q4. Machine-independent answers under a real deadline	9
4	R2: Carrier invention and accumulator discovery	10
4.1	R2.Q1. What finite examples can decide about a minimal carrier	10
4.2	R2.Q2. Polymorphic automata and canonical inputs	12
4.3	R2.Q3. Backward fusion and auxiliary state	12
4.4	R2.Q4. Enumerating recursion motives with parameters	13
4.5	R2.Q5. Using inductive generalization and lemma discovery	14
5	R3: Incremental evaluation of programs with holes	14
5.1	R3.Q1. Higher-order and dependent live evaluation	14
5.2	R3.Q2. Incremental normalization under backtracking	15
5.3	R3.Q3. The cheapest conservative evaluator	16
5.4	R3.Q4. Angelic top-down search and bounded completeness	17
6	R4: Dependent motives, transports, and unfolding	18
6.1	R4.Q1. The scope of finite motive enumeration	18
6.2	R4.Q2. Canonical transport representation	18
6.3	R4.Q3. An unfolding policy informed by existing provers	19
6.4	R4.Q4. A native Lean motive service	19
7	R5: General recursion with explicit obligations	20
7.1	R5.Q1. A capability interpreter and Lean’s recursors	20
7.2	R5.Q2. What top-down angelic recursion must preserve	21
7.3	R5.Q3. A small recursion basis and evidence for coverage	22

8	R6: Contracts, inductive proofs, and proof debt	22
8.1	R6.Q1. Interleaving programs and proofs	22
8.2	R6.Q2. A recognizable class of automatic inductive contracts	23
8.3	R6.Q3. Available induction automation in Lean	24
8.4	R6.Q4. Scheduling proof debt	25
9	R7: Retrieval without forgetting the dependencies	26
9.1	R7.Q1. A useful dependent type abstraction	26
9.2	R7.Q2. Type classes have already been addressed in TYGAR	26
9.3	R7.Q3. Premise selection for data programs	27
10	R8: Ranking and equivalence are different questions	28
10.1	R8.Q1. A defensible ranking objective	28
10.2	R8.Q2. What to merge, and what merely to group	29
11	R9: Learned proposals with reproducible verification	30
11.1	R9.Q1. Accuracy of carrier and helper proposals	30
11.2	R9.Q2. Determinism and auditability beyond receipts	31
12	An integrated implementation plan	32
12.1	One native state, several explicit kinds of evidence	32
12.2	Recommended implementation order	32
12.3	Regression tests tied to the actual failure modes	33
12.4	A benchmark report that can support the claims	34
12.5	What has and has not been resolved	35
A	Question-by-question cross-reference	35
B	Reproducible finite checks	36
B.1	Carrier lower bound	37
B.2	Other checks	37
C	Assumptions and certificate obligations	38
D	Provenance and build instructions	38
	References	39

1 Scope, evidence, and the principal decisions

1.1 What was investigated

This review addresses the source proposal’s numbered questions, not merely its final nine-row summary. There are four questions in R1, five in R2, four each in R3 and R4, three in R5, four in R6, three in R7, and two each in R8 and R9. The question headings below paraphrase their wording while preserving their scope and order. Appendix A maps all 31 questions to their source files and answers. The final source section, `15-experts.tex`, groups these same concerns by the expertise that would be useful; it does not replace the detailed questions.[2]

The repository was inspected at commit `784664ff34e819422510a4d470ab07fe48bb736b`. Its `lean-toolchain` selects Lean 4.34.0. Implementation inspection covered `Core/Types.lean`, `Accept/Gate.lean`, `Engine.lean`, `Native/Transaction.lean`, and the relevant search and residual-evaluation definitions in `Search/Core.lean`. These are source-level observations, not a successful build, test run, or security exploit reproduction. The source proposal reports its own measurements at the earlier commit `33cec8b`; those results must not be silently reassigned to the reviewed snapshot.[1, 4]

The literature search included work published or posted through 21 September 2026. The article cites primary papers, authors’ project documentation, and Lean’s own documentation and source. In particular, the 2026 Canonical-min paper, the ICLR 2026 version of the LeanHammer paper, and recent work on recursive sketch synthesis were considered rather than treating the proposal’s recalled bibliography as current by default.[13, 27, 28] References to the online Lean manual are dated: its latest edition covered 4.35.0-rc2 when consulted, so version-sensitive API advice is anchored separately to the 4.34.0 source.[5, 6]

1.2 How to read the claims

Four kinds of claim are deliberately distinguished. An *established result* is attributed to literature or inspected implementation. A *derived result* is proved in this article, usually as a small mathematical statement that clarifies the design. A *design recommendation* proposes an implementation and states its obligations; it is not presented as already implemented. An *empirical question* is answered with a measurement protocol when the available evidence does not justify a numerical answer.

This is especially important for “complete,” “minimal,” and “verified.” Completeness can be relative to a specified grammar and a terminating decision procedure, or it can mean eventual discovery by fair unbounded enumeration. Neither means that an arbitrary time-limited run decides all Lean inhabitation problems. Minimality requires a fixed ordering and a named space of alternatives. Verification proves the submitted contract, which may be only a finite conjunction of examples; it does not discover or certify an unstated intention.

1.3 The existing evidence supports a targeted rewrite

The proposal reports 278/278 baseline cases and 350/350 signature-corpus cases, but thirteen unscored Church-operation stretch cases remain unresolved. Its edge probes expose missing comparisons and conditionals, unavailable dependent motives, a compilation problem for an accepted recursor term, and insufficient library retrieval. In one historical contract-heavy profile, residual evaluation accounts for 45% of time, while state save/restore accounts for about 1%. These numbers justify studying residual work and search branching before assuming that copying state is the principal cost. They are observations on the proposal’s workload, not universal performance laws.[3]

There is an important consequence for project order. It is unnecessary to solve unrestricted carrier invention before making progress. A bounded carrier grammar, sketches, and a finite observational constraint solver can be delivered while a fair general enumeration lane remains available. Likewise, a complete rewrite of Lean’s evaluator is unnecessary before splitting observations and implementing correct dependency-based invalidation. The first useful milestones should have sharply delimited semantics and independently testable failure modes.

Table 1: Recommended decisions at a glance.

Area	Decision	Boundary that must remain explicit
R1	Persistent whole-state search with dependency-aware scheduling; fixed-cost reference lane.	Goal coupling is semantic, not just shared text. Use $h = 0$ until a lower bound is proved.
R2	Jointly enumerate carriers, algebras, and finishers; use certified finite-state lower bounds.	Finite samples constrain representations; they do not identify a unique intended function.
R3	Incremental typed hole closures, observation-level caches, checked refutations.	A heuristic evaluator may propose; only conservative evidence may eliminate a branch.
R4	Dependency-closed generalization plus bounded motive templates; native Lean induction APIs.	Finite enumeration of a template family is not completeness for all dependent motives.
R5	Explicit recursive-equation IR with well-founded call capabilities.	An interpreter needs a simulation argument; compilation is a separate outcome.
R6	Interleave values and proofs according to dependencies; exploit local algebra invariants.	A proof timeout is neither a counterexample nor a proof of unrealizability.
R7	Relational abstract type hypergraphs and dictionary prerequisites.	Top- k filtering is a budget choice, not a sound proof that other providers are irrelevant.
R8	Fixed program-description cost plus explicit soft preferences and behavioral diversity.	Observational agreement is not definitional or extensional equality.
R9	Optional untrusted proposal transcripts, validated and replayable offline.	Model accuracy on Leant2 carriers is unmeasured; deterministic decoding alone is insufficient.

2 The semantic contract before the algorithms

2.1 Six guarantees that should not share one label

Fix a Lean environment E , a target type T , an optional contract $C : T \rightarrow \text{Prop}$, an axiom policy P , and a grammar $\mathcal{G}$. Treat absent contracts as $C(f) = \text{True}$. A successful synthesis result should identify the original query and contain a closed f and a closed proof π such that

$$E \vdash f : T, \quad E \vdash \pi : C(f), \quad \text{Axioms}_E(f, T, \pi, C(f)) \subseteq P.$$

Universes are part of this statement. Existing universe parameters are rigid unless the user has explicitly requested an instance. Generalizing newly inferred universe metavariables is legitimate only when the resulting declarations are checked at the advertised, jointly generalized types.

This *positive certification* contract is different from five other obligations: that branch pruning never discards a satisfying completion; that enumeration covers a declared grammar; that ranking optimizes the declared objective; that a receipt can be replayed against the same logical dependencies; and that the published program is executable with the advertised behavior. A final kernel check establishes none of those five automatically. It checks the particular declarations that reach it.[5, 4]

Definition 2.1 (Completion semantics). For a typed partial program p in a search state S , let $\text{Comp}_{\mathcal{G}}(S)$ be the well-typed, grammar-respecting substitutions for its open metavariables that satisfy all existing constraints. A completion includes compatible universe assignments and local-context substitutions, not merely textual replacements of question marks.

An unconditional rejection must establish

$$\forall \theta \in \text{Comp}_{\mathcal{G}}(S), \quad \neg C(p\theta). \quad (1)$$

An angelic or unevaluation step usually has a different contract. If it adds a constraint D , it needs at least

$$\forall \theta \in \text{Comp}_{\mathcal{G}}(S), \quad C(p\theta) \implies D(\theta). \quad (2)$$

If D is then proved unsatisfiable over the completion space, rejection follows. Mistaking one possible angelic assignment for a necessary consequence violates this implication. Conversely, proving that a proposed constraint is sufficient for success does not, by itself, justify rejecting all completions outside it; those must remain in another search alternative.

2.2 What the inspected acceptance boundary does, and what to strengthen

The current gate performs synchronous kernel checking through `addDeclCore` with `doCheck := true`, rejects unresolved expression holes and escaped locals, generalizes remaining universe metavariables jointly, and audits the transitive axiom inventory. This is a sound architectural foundation for a small positive-certification boundary. Its public input, however, contains a caller-supplied program type and a caller-supplied proof/type pair. It does not itself take the frozen query or reconstruct the intended contract application. The inspected caller in `Engine.lean` does construct the contract proof type from the extracted subtype value; the observation is an API-hardening recommendation, not evidence that this caller accepts an unrelated proof.[4]

The stronger interface should take the frozen query plus candidate data, reconstruct $C(f)$ inside the boundary, and record that exact proposition in the receipt. It should also retain the checked negative certificate rather than merely returning a negative tag. The inspected contract-impossibility prepass checks its proof under the standard profile and discards the certificate in the returned outcome. A requested stricter policy must govern negative results too, or the result must explicitly say it was established under a different policy.[4]

A second boundary deserves an explicit test. The engine contains an `atProp` transformation and can collect candidates for a universe-specialized target in its classical lane. Such a term is checked at the specialized type, so this is not kernel unsoundness. Nevertheless, an inhabitant of $T[\ell := 0]$ is not generally an inhabitant of the original universe-polymorphic T . If specialization is useful as a suggestion, give it a distinct result carrying its actual type. Do not let a frontend treat it as an ordinary answer to the unspecialized query. This recommendation concerns preservation of the problem statement, independent of the merits of a classical lane.[4]

Finally, `instantiatePartial` explicitly permits an intermediate reduction expression whose metavariable contexts no longer match. Reduction of such an expression is not the same operation as kernel checking a well-typed declaration. Any conservativity proof for early rejection must therefore account for that transformation, or replace it with typed closures. “The kernel reducer was called” is not a sufficient proof of equation (1).[4]

2.3 A result algebra with enough information

A useful result is a structured record, not an undifferentiated success/failure Boolean. The following statuses should be distinct, whether represented as constructors or fields:

Status	Meaning
Verified	A term of the frozen target and a checked proof of its exact contract.
Observed	A typed candidate with recorded execution observations, but without the required contract certificate.
Refuted candidate	A particular candidate fails a checked necessary condition or counterexample.
Impossible	A checked proof that no term satisfies the requested type/contract, under the requested policy.
Bound completed	The declared finite search was covered; unresolved verification obligations are listed separately.
Budget exhausted	Search or checking was interrupted; a continuation and ledger are available.
Publication status	A separate indication of successful declaration export, compilation, and execution checks.

The distinction between the middle two rows is essential. Enumerating every program in a finite grammar does not prove that none meets a universal contract if some proof attempts ended unresolved. Even “grammar exhausted” needs a documented meaning: exhaustion of proof terms up to a proof bound is weaker than refutation of every program in the program grammar. The current `Outcome` and `NegativeKind` already distinguish several negative categories, but do not carry all the evidence and unresolved-survivor information recommended here.[4]

A finite example contract can itself be kernel verified. Thus “Verified” need not mean “universally correct for a hidden intended function.” The user-facing description should name the contract’s scope: for example, “verified against these twelve equalities,” or “verified for all lists.” Merely executing twelve tests without reconstructing their proofs belongs to the weaker observation status.

2.4 Corrections that affect the research plan

The detailed answers prove or explain several corrections. Reversal is a right fold over a list result carrier when the step may append; an efficient function carrier is an optimization, not a semantic necessity. A count of open goals is not generally an admissible remaining-cost bound. Smyth already supports higher-order examples in its implementation and removes trace completeness. Hoogle+ already represents type-class requirements through dictionaries. The transport example $n + 0$ should be distinguished from $0 + n$ under Lean’s standard recursive definition of addition. Sorting candidates after a parallel timeout does not determine which candidates reached the set. A finite Kripke countermodel for a reified propositional problem proves nonvalidity in that reified calculus, not automatically a Lean theorem negating an arbitrary contextual target.[2, 14, 17, 5]

These are not reasons to abandon the proposal. They identify where a useful engineering experiment can replace a falsely universal theorem, or where an existing technique can replace an unnecessarily open-ended research dependency.

3 R1: Search control and shared metavariables

3.1 R1.Q1. Search without duplicating all dependent work

Question, paraphrased. Can shared-metavariable goals be organized more effectively than copying entire alternatives or solving every goal sequentially? Can independence be detected and reused?

Answer. Yes: use a persistent whole-state frontier, but represent constraints inside each state as a dependency hypergraph. Independence should be a certified special case of this representation, not the default assumption. Aesop supplies an established best-first, rule-indexed reference design; it does not make arbitrary shared obligations independent.[11]

The vertices are outstanding term and universe metavariables. A factor contains an obligation together with every variable on which its meaning can depend. The support must include the target's type, types and values of relevant local declarations, delayed assignments, deferred instance problems, and residual contract observations. A contract $f(0) + g(0) = 1$, for example, couples the bodies of f and g even when their Lean types do not share a metavariable. A common immutable local parameter does not itself couple two searches; a mutable variable in its type can.

After an assignment, instantiate the affected factors and update their supports. Components can split when a shared variable becomes fixed. A plain union-find structure is therefore insufficient on its own: it efficiently merges but does not represent this splitting. Recompute small affected components, or use a persistent adjacency representation with a work queue. Start with conservative overapproximations. False dependency edges cost performance; missed dependency edges can invalidate independent pruning.

Proposition 3.1 (Factorization criterion). *Suppose the remaining completion variables partition into $V_1, \dots, V_k$. Each variable's domain, including typing and grammar restrictions, depends only on variables in its own block and fixed parameters. Suppose every remaining constraint belongs to one block, and no permitted subsequent refinement introduces cross-block constraints. Then a global completion exists exactly when every block has a completion. If total completion cost is a sum of block costs, a minimum-cost global completion is assembled from minimum-cost block completions.*

Proof. Restrict any global completion to each V_i to obtain the forward implication. Conversely, take a completion for each block and unite the substitutions. Their domains are disjoint, their typing dependencies are local or fixed, and every constraint is satisfied within one block, so the union is well typed and satisfies all constraints. Additivity gives the minimum-cost assertion by replacing any nonminimal block of a proposed optimum. $\square$

The final hypothesis about future refinement matters. A rule that later introduces a helper shared by two blocks changes their independence. Either forbid this after a block has been sealed, or rejoin the blocks. Similarly, a requirement to use a provider at most once, a global syntax-size budget, or a nonlinear ranking objective introduces additional coupling. A size budget can be handled by storing each component's cost-indexed solution stream and combining streams, rather than taking just one solution per component.

For cross-branch memoization, store a *typed recipe*: an abstracted local telescope, normalized target, residual constraints, grammar identity, and a candidate term expressed over fresh formal parameters. Instantiate that recipe with fresh metavariables and recheck it in the receiving context. Do not cache a raw assignment containing an old `MVarId`. Negative cache entries additionally need the precise exhausted bound and a proof that the reused search space is covered. A failed bounded proof attempt is not a reusable impossibility fact.

This can be implemented incrementally. Keep today's whole-continuation rollback discipline. First use the graph only to prioritize informative goals. Next memoize closed successful recipes.

Only then allow independently certified components to commit without global backtracking. This sequence earns performance without simultaneously changing every correctness argument.

3.2 R1.Q2. An admissible remaining-cost bound

Question, paraphrased. What lower bound is correct when solving one goal can instantiate or close others?

Answer. The universally safe starting point is $h(S) = 0$, with nonnegative fixed transition costs. Counting goals is not generally admissible. Consider two residual constraints $n = 0$ and $n + 1 = 1$ sharing an unknown n . In a cost model where assigning $n := 0$ costs one and reflexive propagation is free, one charged action closes both. A proposed bound of two exceeds the actual remaining cost of one. Giving exact-local steps zero cost creates additional counterexamples unless those steps are completely normalized before estimating cost.

Let $c^*(S)$ denote the least additional cost of any successful completion. For a lower bound ℓ_i attached to obligation i , the desired statement is $\ell_i \leq c^*(S)$, not merely that ℓ_i estimates the effort of solving that obligation in isolation. Within one coupled component,

$$h(S) = \max_i \ell_i$$

is valid whenever each individual inequality is proved. Summing these bounds is not valid without a disjoint-charge argument. For components satisfying theorem 3.1, with additive disjoint costs, independently established component bounds may be summed.

A more informative systematic approach is a *relaxed search problem*. Erase selected constraints, add possible transitions, and assign each abstract transition no greater cost than the concrete transition it represents. If every concrete successful path projects to an abstract successful path, the abstract shortest-path distance is a lower bound. The abstraction must retain one-to-many propagation: charging separately for two goals that one concrete action discharges defeats admissibility. This method is a design framework, not a claim that a cheap, strong abstraction has already been found for Leant2.

With $h = 0$, uniform-cost search is optimal only under the usual operational assumptions: the graph faithfully represents the intended completion space, all costs are fixed and nonnegative, and enough finite-cost work can be completed before the solution. Zero-cost cycles or infinitely many zero-cost alternatives can prevent progress. Use canonical administrative normalization plus deduplication, or a positive syntactic grade for every genuine expansion. If an admissible but inconsistent heuristic is later added to a graph search, allow reopening states when a better g value is found.

3.3 R1.Q3. Learning search costs without changing the promise

Question, paraphrased. Can just-in-time learning safely guide search and ranking, and how should budget effects be controlled?

Answer. Separate three quantities: a fixed program objective, accumulated charged work, and a learned proposal priority. They answer different questions. Probe demonstrates learning a synthesis distribution from partial successes; it does not establish that a deadline-limited learned run returns the same candidates as a fixed-cost exhaustive run.[16]

For Leant2, keep a stable objective such as a description length over the declared grammar. Use learned priorities to order a second frontier, or to break ties inside completed fixed-cost bands. Do not retrospectively decrease an edge cost while continuing to claim that the original A^* proof applies. If the objective really changes, start a new optimization epoch, recompute affected bounds, and label the result with the new objective.

A deterministic portfolio can allocate a fixed fraction of logical work to a reference enumerator and the rest to adaptive search. Its important property is fairness, not a universal speedup theorem. If every finite reference prefix eventually receives work, no finite reference derivation is permanently starved. At a finite deadline, however, different priorities can still produce different answers. The receipt should therefore include the objective version, learning-state digest, ordered decisions, and completed reference bound.

Selection and final presentation should also remain separate. A learned heuristic can search aggressively for useful programs while final ranking applies a fixed, transparent ordering to the verified set. That gives stable ranking *of a given set*, not stability of the set itself. A best-candidate guarantee requires a frontier lower-bound certificate against the same final objective; a 400 ms grace interval does not provide it.

3.4 R1.Q4. Machine-independent answers under a real deadline

Question, paraphrased. Can wall-clock-limited results be made independent of machine speed without a calibrated node budget?

Answer. Not in the unrestricted form suggested. A deterministic logical trace has prefixes $S_0, S_1, \dots$. Place the first new candidate after step j . Two machines can complete respectively $j - 1$ and j steps before the same deadline. Unless the algorithm withholds the additional candidate or disregards the deadline, their results differ. Arbitrarily slow machines make any nontrivial fixed-deadline guarantee impossible.

The attainable guarantee is *prefix determinism*. Specify a deterministic logical work sequence, return the exact completed prefix, and allow continuation from it. Charge at stable points such as rule applications, unification attempts, and verified observation reductions. A single “node” is often too coarse: a node can invoke a very expensive normalizer or prover. Maintain separate logical counters and physical time, with interruptible subbudgets for costly services. Wall time decides where the trace stops, not what that trace means.

For parallel execution, give tasks stable identifiers and deterministic work allotments. Commit completed epochs in task order. At a deadline, report the last fully committed epoch and explicitly separate opportunistic extra results. Sorting all candidates that happened to finish cannot undo the race that determined set membership. Similarly, resuming three two-second runs need not reproduce one six-second run, because replay and process startup consume time. Resuming to the same *logical work prefix* can reproduce it.

Luby-style restart schedules have a theorem for repeated runs of Las Vegas algorithms under a runtime-distribution model. They do not automatically optimize heterogeneous deterministic synthesis lanes; restarting the same deterministic prefix repeatedly can simply waste work.[22] Use a resumable, fair geometric allocation as the default, and treat restart randomization as a separately measured policy. Recent snapshotting work concerns avoiding expensive re-elaboration in external tactic-search pipelines; its cost regime is not the same as Leant2’s already-persistent in-process state.[23, 3]

4 R2: Carrier invention and accumulator discovery

4.1 R2.Q1. What finite examples can decide about a minimal carrier

Question, paraphrased. Is there a constructive procedure that finds a smallest carrier type from a grammar, or proves that no such carrier fits the examples?

Answer. There is a complete constructive answer for an explicitly finite problem. There is no corresponding general decision procedure merely because the *type grammar* is finite. The carrier, step, initialization, and observation map must be distinguished:

$$q([]) = z, \quad q(a :: x) = s(a, q(x)), \quad h = o \circ q, \quad (3)$$

where $z : K$, $s : A \rightarrow K \rightarrow K$, and $o : K \rightarrow T$. Minimal state cardinality, smallest syntax for K , cheapest executable realization, and best match to an intended function are four different objectives.

If K ranges over a finite list of types but step bodies range over arbitrary Lean terms, type and contract synthesis have not become a finite problem. If o is unrestricted and $K = \text{List } A$ is allowed, every function h factors through the identity list fold: choose $z = []$, $s = \text{cons}$, and $o = h$. Consequently, a semantic obstruction to writing h itself as a fold cannot automatically rule out a fold followed by an arbitrary finisher.

Here is a precise finite version. Fix finite alphabets A , finite output sets T , explicitly enumerated finite carriers K , and permit every total table for s and o . For each K , enumerate its initial state, transition table, and output table; evaluate all sample words. Order carriers by a declared cost and return the first satisfying representation. This terminates, and failure is a genuine finite-state impossibility result for these assumptions. Alternatively encode the same problem as a finite constraint satisfaction problem, with variables for the state of each observed suffix and the entries of the two tables. A complete Lean-native backtracking solver suffices; an external solver is optional, not necessary.

To translate this answer back to syntax, require either an emitter that realizes every table using the carrier's finite enumeration and equality, or an explicit finite grammar for the initialization, step, and finisher. The latter can be exhaustively searched by total syntactic size. A carrier table that is extensionally realizable but too large for the step grammar does not solve the bounded syntactic problem. For arbitrary Lean checks, resource exhaustion must remain unknown; a decidable finite fragment or a completed certificate checker is needed for a definitive negative answer.

The exact semantic lower bound

The following formulation makes the automaton analogy precise without confusing samples with a complete specification. It is a direct algebraic argument, related to classical fold characterization and automata learning.[18, 19]

Definition 4.1 (Continuation equivalence). For $h : \text{List } A \rightarrow T$, define

$$x \sim_h y \iff \forall u : \text{List } A, h(u ++ x) = h(u ++ y).$$

These are *left* continuations because q in equation (3) is a right fold.

Theorem 4.2 (Minimal reachable state quotient). *Every factorization equation (3) has at least as many reachable states as $\sim_h$ has equivalence classes. Conversely, the quotient by $\sim_h$ supports a factorization of h . Thus it has minimum reachable-state cardinality among arbitrary set-theoretic right-fold representations with an output map.*

Proof. If $q(x) = q(y)$, induction on u gives $q(u ++ x) = q(u ++ y)$. Applying o shows $x \sim_h y$. Hence the map from a reachable state $q(x)$ to the equivalence class $[x]$ is well defined and surjective.

For the converse, use states $[x]$, initial state $[\]$, transition $s(a, [x]) = [a :: x]$, and output $o([x]) = h(x)$. Output is well defined by choosing $u = [\]$. The transition is well defined because $u ++ (a :: x) = (u ++ [a]) ++ x$. Induction gives $q(x) = [x]$, so $o(q(x)) = h(x)$. $\square$

For infinite cardinal comparisons, the theorem is stated in ordinary classical set theory with choice; its finite lower-bound applications need only elementary finite counting. It constructs a mathematical quotient, not a computable presentation of that quotient from finitely many examples. In particular, it does not identify a preferred Lean type among isomorphic carriers. Infinite types such as Nat also make raw cardinality a poor proxy for useful state structure: encodings can hide considerable computation in the transition and finisher.

Finite observations do nevertheless yield certified lower bounds. Choose suffixes x_i and prefixes u_j for which outputs are specified. Join two suffixes by an edge when some shared observed prefix gives different outputs. Any consistent carrier assignment must color this incompatibility graph: adjacent suffixes need distinct states. A witnessed clique of size r proves that fewer than r states cannot work. The chromatic number is a stronger lower bound; transition consistency can strengthen it further.

Do not count distinct *partial* observation rows as distinct states. Compatibility of incomplete rows is not transitive. For example,

$$(0, ?), \quad (?, 1), \quad (1, 1)$$

has compatible first–second and second–third pairs, but incompatible first–third entries. Unknown cells are not a new output value. Nor does enumerating the values constructible up to depth d provide an upper bound on all values a type can hold: it gives a lower bound on its discovered inhabitants. Cardinality pruning requires an actual upper-bound proof.

Example 4.3 (A four-state requirement from fifteen observations). Let $A = \{a, b\}$, and let $h(x)$ say that x contains both letters. The suffixes $[\]$, $[a]$, $[b]$, $[a, b]$, observed under prefixes $[\]$, $[a]$, $[b]$, give the rows

x	$[\]$	$[a]$	$[b]$
$[\]$	0	0	0
$[a]$	0	0	1
$[b]$	0	1	0
$[a, b]$	1	1	1

so all four suffixes are pairwise incompatible. All observations involved are words of length at most three. No carrier with at most three states fits them. A pair of Booleans works: record whether a has appeared and whether b has appeared; combine the two bits for the output. The supplied script independently exhausts every one-, two-, and three-state transition/output table with a fixed initial-state label and finds no survivor.

For Leant2 this suggests a dual output from carrier inference: a proposed carrier plus code, and, when available, a small distinguishing-context certificate excluding smaller finite carriers. That is more informative than a heuristic carrier guess and much cheaper to check than rerunning the whole search.

4.2 R2.Q2. Polymorphic automata and canonical inputs

Question, paraphrased. Does the state-machine perspective extend to polymorphic functions, perhaps using positions or equality patterns as the alphabet?

Answer. Yes, under an explicit relational-parametricity assumption for the program fragment and its providers. It is not valid for every term allowed by Lean’s standard axiom profile. The literature on testing polymorphic properties establishes reductions to suitable monomorphic instances for specified classes of parametrically polymorphic properties; its hypotheses are substantive.[20]

Suppose $f_A : \text{List } A \rightarrow \text{List } A$ is natural in A :

$$f_B(\text{map } e \ x) = \text{map } e \ (f_A(x)) \quad (e : A \rightarrow B). \quad (4)$$

Let $c_n = [0, \dots, n-1] : \text{List}(\text{Fin } n)$. For any list x of length n , let $e_x : \text{Fin } n \rightarrow A$ select its entries. Then

$$f_A(x) = \text{map } e_x \ (f_{\text{Fin } n}(c_n)).$$

Thus a list of output positions describes every behavior at a fixed input length, including duplication, deletion, and reordering. The case $n = 0$ is included: the finite type has no inhabitants, so its output list is empty. Agreement on canonical inputs through length N , together with equation (4) for both functions, proves agreement on all types and all lists of length at most N . It does not prove agreement at greater lengths.

For contrast, in classical Lean one can define a noncomputable family that returns its input when A is a subsingleton and returns the empty list otherwise, using classical decidability of $\text{Subsingleton } A$. Mapping from a two-element type to a singleton shows failure of equation (4). The type $\forall A, \text{List } A \rightarrow \text{List } A$ alone therefore does not license the canonical-position argument.

If the language exposes lawful decidable equality on elements, arbitrary maps are no longer the relevant relation-preserving morphisms. Equality patterns matter: canonical inputs should realize partitions of positions, not only the all-distinct partition. Extra operations, orders, algebraic dictionaries, or distinguished elements require richer test structures and preservation laws. A user-supplied BEq without laws cannot be silently identified with propositional equality.

The practical first implementation should certify parametricity for a deliberately small grammar of constructors, eliminators, and explicitly approved polymorphic providers. Until that exists, positional probes remain useful heuristics for presentation and counterexample discovery, but must not justify permanent merging or impossibility claims. Symbolic state representations may keep a finite control tag together with stored positions or elements; a finite abstract alphabet does not imply that the concrete carrier itself is finite.

4.3 R2.Q3. Backward fusion and auxiliary state

Question, paraphrased. Can examples be worked backward to discover a carrier, and is the method complete for useful classes?

Answer. Use backward reasoning to propose a *commuting algebra*, not to infer a unique hidden auxiliary function from samples. Write an unknown summary q and seek equations

$$q([]) = z, \quad q(a :: x) = s(a, q(x)), \quad o(q(x)) = h(x).$$

When q is a tuple $(q_1, \dots, q_m)$, failed closure obligations often reveal which additional summary component is missing. The state grammar and the equations for all components should be searched together. Auxiliary state is useful precisely when it makes the local transition inexpensive and expressible.

For natural-number Fibonacci, the summary

$$q(n) = (F_n, F_{n+1}), \quad q(0) = (0, 1), \quad s(x, y) = (y, x + y)$$

turns two-step history into a one-step transition. Its invariant is proved by induction from the Fibonacci equations. For maxima of possibly empty lists, an optional stored element avoids inventing an element of an arbitrary empty type. For a tail-recursive helper, a strengthened equation such as $go(x, a) = h(x) \star a$ simultaneously identifies the accumulator and the induction invariant.

Reversal clarifies what the optimization does and does not establish:

$$\text{reverse} = \text{foldr } (\lambda a \ r. r ++ [a]) \ [].$$

This is already a fold with carrier List A. Appending to the end repeatedly gives quadratic list traversal in the usual immutable singly linked representation. A function carrier admits

$$q([]) = \text{id}, \quad q(a :: x) = \lambda y. q(x)(a :: y), \quad o(k) = k([]).$$

The invariant $q(x)(y) = \text{reverse}(x) ++ y$ proves correctness. Under constant-time cons and closure construction, it avoids repeated append traversal. Alternatively, changing to a left-fold scheme also permits a linear reversal with a list carrier. Neither observation proves that reversal semantically requires a higher-order carrier.

For a fixed finite alphabet and finite carrier, the exhaustive procedure in R2.Q1 is complete for all finite transition tables, including those suggested by backward reasoning. For a finitely specified polynomial datatype, fixed carrier, and bounded algebra grammar, constructor-by-constructor enumeration is complete relative to that grammar. Once carrier types, helper functions, semantic invariants, and unrestricted finishers can all be invented, no comparable terminating general guarantee follows. A useful promise is fair enumeration by a combined size bound, with backward fusion changing the order and providing pruning certificates when available.

4.4 R2.Q4. Enumerating recursion motives with parameters

Question, paraphrased. Can primitive-recursion motives be enumerated completely and efficiently without recreating the natural-number search explosion?

Answer. Completeness and efficiency need separate statements. For a finite grammar of motives, local parameters, and auxiliary carriers, enumerate by a *joint* grade rather than running an unbounded motive search before body synthesis. Use a cost such as

$$|K| + |M| + |z| + |s| + |o| + \lambda \# \text{recursive schemes}.$$

The notation denotes syntax size, not state cardinality. Fairly increasing this grade eventually considers every term in the grammar; it does not bound the time needed at a given grade.

Start with the result family, function carriers over existing parameters, small products, and options. For an indexed datatype, abstract the indices in the result family and generalize dependency-closed context subsets, as detailed in R4. For ordinary Nat.rec, try outermost recursion first and charge nested recursion explicitly. The uncontrolled multiplication of recursion choices in every natural-number hypothesis is a search-order problem, not an argument for permanently removing all natural-number recursion.

A bound on carrier syntax alone is insufficient because a function carrier can conceal arbitrarily large step code. Conversely, limiting recursion to an outermost scheme is an effective first tier but is incomplete for the unrestricted grammar. Make the tier visible in the completed-bound metadata and eventually expand it in the reference lane. No theorem turns arbitrary higher-order motive invention into a cheap finite preprocessing step.

4.5 R2.Q5. Using inductive generalization and lemma discovery

Question, paraphrased. What can techniques from inductive theorem proving contribute to carrier and helper invention?

Answer. Transfer the *shape of the obligation* as much as the final theorem-proving machinery. If an induction step is blocked because an accumulator has changed, generalize that accumulator before induction. If the step repeatedly creates an unexpressed statistic, propose that statistic as a component of the carrier. If normalization exposes an associative combination, propose a helper invariant that distributes over it.

Theory exploration systems such as Hipster already combine conjecture generation with induction-based proof attempts, including discovery of lemmas missing from a current proof.[21] The Leant2 adaptation should make this a bounded service: extract terms and type-correct anti-unifications from the failed preservation goal; propose a small number of generalized equations; refute bad proposals using concrete instances; then prove surviving equations before using them as rewrite rules or search cuts. A theorem that is useful only after a helper is introduced should be linked to that helper's synthesis task, not asserted as an oracle.

Rippling-style annotations can be represented as search metadata describing how a goal differs from its induction hypothesis. They suggest which term position to abstract or which lemma might move the difference outward. The resulting suggestion is not a theorem and should not force a single irreversible generalization. In a native Lean implementation, the first version can be much smaller than a full rippling engine: recognize accumulator arguments, retained input tails, and constructor-preservation equations, then try a bounded library of invariant schemas. Measure the incremental solved tasks and the extra proof debt, not merely the number of conjectures generated.

5 R3: Incremental evaluation of programs with holes

5.1 R3.Q1. Higher-order and dependent live evaluation

Question, paraphrased. How far does live bidirectional evaluation already extend, and where does a Lean adaptation become difficult?

Answer. Higher-order support is not wholly missing from the starting literature. Core Smyth restricts its formal example language for simplicity, but the implementation supports higher-order examples and polymorphism. It also deliberately eliminates the requirement that examples describe every recursive call. These are explicit distinctions in the paper, not speculative extensions.[14]

The appropriate Lean object is a *typed hole closure*, conceptually

$$(m, \rho, A, \kappa),$$

where m identifies the unknown expression, ρ records the substitution for its local telescope, A is its resulting type, and κ records the suspended eliminations or observations. The same hole applied in different environments is one shared unknown with several constraints, not several independently fillable holes. A result of evaluating $\text{map } ?f [a, b]$ can preserve the list spine while retaining two closures for the same f .

Unevaluation should return a *relation on possible completions*. From $\text{cons } ?x \ ?xs = a :: ys$, constructor injectivity supplies necessary constraints on both holes. From $g(?x) = y$ for an arbitrary g , no unique value for x follows. Retain the equation, use a proved preimage rule, or branch over alternatives whose union covers the possibilities. A higher-order observation can

constrain a function at specified arguments, but equality on those arguments is not extensional equality of functions.

Dependency adds two further issues. The type of a closure can itself depend on an unknown index, and values observed under different index substitutions may require transports before comparison. Moreover, a proof argument can affect definitional reduction even when it is erased at runtime. A representation that simply drops proof fields or treats all casts as identities is not automatically a correct symbolic evaluator for Lean expressions. Use typed transport nodes with explicit endpoints, and let certified simplification remove them when possible.

None of the examined live-evaluation or recursive-sketch papers supplies a drop-in conservativity theorem for Leant2’s full native Lean expression language, universe handling, and delayed assignments. The first implementation should therefore support a sharply specified fragment: constructors, applications, lambdas, let bindings, projections, selected recursors, and named pure primitives with checked semantics. Outside that fragment, suspend or fall back to the existing checker. Lack of an evaluation rule is *unknown*, not a refutation.

5.2 R3.Q2. Incremental normalization under backtracking

Question, paraphrased. Is there an existing model for waking computation when holes are assigned, and how should memoization survive rollback?

Answer. Use dependency-triggered suspended computations and persistent cache versions. A particularly relevant recent reference is Canonical-min: its 2026 paper attaches postponed type-checking constraints to metavariables and resumes them on assignment, with push/pop behavior during search. It is a compact inhabitation/unification engine, not a verified incremental evaluator for all Lean programs; its soundness and completeness arguments concern the paper’s calculus, and their full mechanization is left as future work.[13]

For Leant2, each evaluated observation should return its result together with a *read set*. This must include assignments already read, not just holes on which reduction stopped. Suppose observation $m = 1$ is false in a branch assigning $m := 0$. After rollback, a sibling assigns $m := 1$. Reusing the old Boolean because it has no remaining blockers rejects the correct sibling. A cache keyed solely by an expression and the set of currently unassigned holes is insufficient.

A defensible key consists of an expression/closure identity, environment and reduction-policy identity, and the versions of every mutable dependency read. Versions must not suffer an ABA problem: a numerical assignment counter reset by rollback can accidentally identify two different branches. Persistent state identities or monotonically fresh assignment stamps solve that problem. Local instance context, universe assignments, delayed-assignment recipes, and transparency settings belong in the dependency model where they influence evaluation.

Cache entries should be immutable. Backtracking restores a root into a persistent cache graph, or retains entries whose validity predicates are checked against the current snapshot. Watchers index observations by the variables they read, so an assignment schedules only affected observations. A mutation to an unrelated goal should not invalidate all observations; a new observation added by counterexample generation must get its own cache identity.

The implementation path is deliberately modest. First cache closed observation results under an exact query/environment identity. Next cache residuals within one branch. Then enable cross-branch reuse only after assignment-version and delayed-substitution tests pass. Record hits, invalidations, recomputation work, and retained memory separately. A large hit rate is not a benefit if closure retention causes memory growth that dominates the saved reductions.

5.3 R3.Q3. The cheapest conservative evaluator

Question, paraphrased. How can a faster evaluator be made provably safe for pruning, and is Lean4Lean a practical starting point?

Answer. The cheapest *trustworthy integration* is usually a fast proposer with a small checked negative-evidence path, not immediate replacement of the kernel reducer. Lean4Lean provides a Lean implementation of an external typechecker and an ongoing metatheoretic verification program; the paper does not justify assuming that extracting arbitrary reduction routines yields a fully proved evaluator for open Leant2 states.[24]

There are three sensible deployment levels. At the first, fast evaluation affects scheduling only. At the second, a fast failure proposes a compact contradiction certificate, which Lean checks before pruning. At the third, the evaluator has a proved simulation or abstract-interpretation theorem for its supported fragment, and its returned negative fact can be trusted by that theorem. Differential testing is valuable at all levels, but cannot replace the theorem required at the third.

Proposition 5.1 (A sufficient certificate for safe residual pruning). *Suppose a well-typed partial program is represented as an abstraction over a dependency-ordered telescope of holes. Suppose the checker validates, under the frozen environment and policy, a term of*

$$\forall \vec{h}, D(\vec{h}) \rightarrow \neg C(p[\vec{h}]),$$

where every allowed completion satisfies D . Then rejecting the corresponding branch preserves every contract-satisfying completion of the grammar.

Proof. Substitute the hole values and their typing dependencies from any allowed completion into the checked theorem. Apply its D assumption, which the completion satisfies. The conclusion is the negation of that completion’s contract, so no satisfying completion is removed. $\square$

This criterion explains both why neutral-hole reduction can be safe and why ill-scoped reduction intermediates deserve special attention. If the evaluator only performs substitution-stable reductions on a well-typed abstraction, a proof of a constant Boolean result can often be reconstructed cheaply. If it has first erased dependencies, guessed equalities, or unfolded a differently scoped hole, the theorem’s premise has not been established. The current reduction-only delayed-assignment expansion should be tested and justified specifically at this boundary.[4]

A useful result datatype is conceptually:

```
EvalResult :=
  value      (typedValue, readSet)
| suspended (typedClosure, readSet)
| rejectHint (diagnostic, readSet)
| refuted    (checkedCertificate, readSet)
| resourceExhausted
```

Listing 1: Proposed interface, not compiled Lean code.

Only `refuted` eliminates a branch in the conservative lane. A `rejectHint` can trigger a cheaper replay with native reduction or move the branch later in a heuristic lane; it cannot support a negative completeness claim.

Observation extraction also needs a contract. To refute C , it is enough to check a necessary condition $C \Rightarrow C_i$. To accept by collecting proofs of C_i , one needs the converse implication from their conjunction to C . Splitting conjunctions is straightforward; splitting an arbitrary Boolean combinator or higher-order library predicate requires its actual semantics. Do not replace a user-defined Boolean equality test by mathematical equality unless the required law is proved.

Conversely, a contract explicitly written using that Boolean test must be checked with that exact dictionary, even if the dictionary is unconventional.

The test suite should stress open terms, repeated hole applications, universe substitutions, indexed constructors, casts, delayed assignments, local instances, and sibling rollback. Agreement on a hundred thousand closed constructor terms could miss the principal failures of an incremental search evaluator. A tested fragment with conservative fallback is a useful deliverable long before a fully verified universal evaluator.

5.4 R3.Q4. Angelic top-down search and bounded completeness

Question, paraphrased. Can a top-down angelic evaluator obtain useful termination or completeness guarantees comparable to bottom-up recursive synthesis?

Answer. Yes, relative to a finite concrete search space and a sound approximation of its recursive behavior. Burst shows how angelic execution can be combined with strengthening and resynthesis; its bottom-up representation shares sets of candidate programs rather than independently committing a body at each call.[15] A top-down implementation must preserve that global sharing by constraints on common holes.

Maintain a symbolic application record for a recursive call or unknown function application. Calls known to have equal inputs must have compatible outputs. Unequal-looking symbolic inputs must not be assumed unequal without justification. An observed value for a recursive call is an assumption derived from the submitted specification, when applicable, not the result of executing the unknown implementation. Calls outside the observation set may remain unconstrained. That is a safe overapproximation, though possibly a weak one.

Theorem 5.2 (Finite relative completeness of checked refinement). *Let F be a finite set of concrete well-typed candidates. Assume candidate verification terminates and decides the specified finite observations. Suppose every refinement preserves all candidates in F satisfying those observations, and every unsuccessful completed iteration removes at least its selected failing concrete candidate. If selection does not permanently omit any remaining candidate, the procedure terminates with a satisfying candidate or correctly reports that none exists in F .*

Proof. A satisfying candidate is never removed. Every unsuccessful completed iteration strictly decreases the finite remaining set, so there are at most $|F|$ such iterations. If the set becomes empty, no satisfying candidate originally existed; otherwise selection and deciding verification eventually produce one. $\square$

This elementary theorem is useful because it exposes hidden assumptions. A finite set of guessed angelic return values is an *underapproximation* unless it is known to cover all realizations in F . Unsatisfiability over such a guessed set does not refute the unrestricted branch. A timeout inside the angelic solver does not establish unsatisfiability. Rejecting one failed angelic assignment does not reject all assignments. For an infinite grammar, fair enumeration and eventual successful checking provide a semidecision guarantee, not termination on failure.

The proposed Leant2 design should combine top-down typed holes with a small bottom-up pool of reusable closed expressions and behavioral summaries. Keep the typed realizations behind every summary. As observations grow, split the summaries or regenerate alternatives rather than discarding their provenance. The essential issue is not whether the outer loop is called top-down or bottom-up; it is whether its approximations include all relevant realizations and whether refinement facts are valid for every realization they eliminate.

6 R4: Dependent motives, transports, and unfolding

6.1 R4.Q1. The scope of finite motive enumeration

Question, paraphrased. Is abstracting subsets of index occurrences complete and terminating, and does the resulting family cover accumulator generalization?

Answer. It is a finite, useful candidate generator, not a complete account of motive synthesis. Fix the goal syntax, a finite set of nonoverlapping candidate occurrences, and a finite collection of context-generalization choices. Enumerating their subsets terminates as a syntax-generation operation. Some generated abstractions are ill typed; some cannot be checked within the available resources. Completeness therefore means only that every member of this explicit template family is visited, with checking outcomes recorded honestly.

For vector mapping, the relevant step is concrete. From a goal of the form

$$\forall n, (A \rightarrow B) \rightarrow \text{Vec } A \ n \rightarrow \text{Vec } B \ n,$$

keep the element mapping available and choose the motive

$$M(n, x) = \text{Vec } B \ n.$$

The cons branch then receives an induction hypothesis at the tail's length, rather than an obligation that still mentions the original fixed length. The index abstraction makes the recursive result useful.

Generalization must preserve the dependency order of local declarations. If a local value has type $\text{Fin } n$, abstracting n while retaining the value as though its type had not changed is ill scoped. Revert dependent locals together, reintroduce the generalized telescope, and retain the corresponding substitution. For repeated or compound indices, first introduce fresh variables with suitable equalities, then perform the dependency-aware elimination. Arbitrarily replacing selected text occurrences is not enough.

The family is not closed under accumulator invention. A motive may require a fresh product component, a new parameter, or a strengthened invariant that does not occur as a subexpression of the target. Combine context abstraction with the bounded carrier grammar from R2 and charge both the extra telescope and its implementation. "All maximal occurrence subsets" and "all useful motives" are different sets. A fair size-graded union of expanding template families is a defensible search promise; a fixed finite family cannot justify an unrestricted negative result.

6.2 R4.Q2. Canonical transport representation

Question, paraphrased. Should casts be normalized through heterogeneous equality or setoids, and can they be lowered only at the end?

Answer. Keep explicit, typed transports in the intermediate representation. Use heterogeneous equality where it expresses a genuinely heterogeneous relation, not as a blanket replacement for well-typed terms. A setoid changes the equality problem being solved and requires every relevant operation and contract to respect that relation; it is not a generic substitute for Lean equality.

A transport node should retain its source type, destination type, equality proof, and transported term. Normalize obvious identities and adjacent compositions using checked lemmas. For example, if $p : A = B$ and $q : B = C$, transport by p followed by transport by q agrees propositionally with transport by their composite. This is proved by equality elimination on p and q . It supports an optimization, but does not entitle an untyped simplifier to discard endpoints.

Lean’s proof irrelevance identifies proofs of the same proposition. It does not equate arbitrary types or erase the typing role of their equality proofs. Runtime proof erasure, definitional equality of kernel terms, and equality of typed program meanings must remain separate layers. Charging very little for administrative proofs can improve ranking, but give generated syntax a positive enumeration grade or canonical representation so transport loops do not create infinitely many zero-cost alternatives.

There is a small correction to the proposal’s example. With the standard definition of natural addition, $n + 0$ is definitionally n ; the opposite orientation $0 + n$ is justified by `Nat.zero_add`. The 4.34.0 library proves the latter by recursion. Thus `Vec A (n+0)` is not the intended example of an unavoidable propositional cast; `Vec A (0+n)` is the useful contrast.[7]

6.3 R4.Q3. An unfolding policy informed by existing provers

Question, paraphrased. Should definitions be unfolded up front, kept opaque, or expanded on demand?

Answer. Use a demand-driven transparency ladder, with a small explicit normalization vocabulary at its first level. Existing systems illustrate both useful extremes. Canonical’s translation unfolds aliases of function types and reducible definitions, recursively translates definition dependencies, and explicitly warns that this can break abstraction barriers and expand the problem dramatically. CoqHammer’s documented `sauto` options distinguish heuristic unfolding from unconditional unfolding of selected constants.[12, 25]

For Leant2, first expose the head needed for introduction, constructor selection, or elimination. Next unfold a small set of relevant reducible aliases and the definitions used by the selected recursor or contract observation. Escalate only when a blocked unification or normalization request identifies a useful dependency. Avoid eagerly expanding every implementation in a retrieved provider’s transitive closure.

A reduction policy is part of search state and cache identity. A negative result under a restricted transparency or provider policy must not be reused as though it ruled out a broader search. At the same time, keep ordinary provider names available as compact terms: unfolding a definition for reasoning does not require removing it as a synthesis component. This separates semantic normalization from presentation and prevents the grammar from duplicating every term through arbitrarily many expanded aliases.

On-demand unfolding is a recommendation, not a demonstrated optimum. Compare it with a fixed policy on the indexed-vector, arithmetic, and large-library probes. Measure exposed declarations, normalization steps, failed unifications, cache invalidations, and verified results. A policy that lowers unification time but multiplies candidate expressions may be a regression overall.

6.4 R4.Q4. A native Lean motive service

Question, paraphrased. Does Lean expose enough machinery to try motive abstractions cheaply and report useful failures?

Answer. Yes. In the pinned Lean 4.34.0 source, `Lean.Meta.mkRecursorAppPrefix` explicitly constructs the recursor application through its motive, abstracting the major premise and indices from the goal. `getMajorTypeIndices` checks the index structure, while `Lean.MVarId.induction` handles reverting and reintroducing dependencies and returns `InductionSubgoal` records with substitutions. These are concrete building blocks, not a conjectural future API.[6]

Their preconditions explain the wrapper Leant2 needs. The index extractor expects suitable variable indices and rejects repeated or malformed occurrences; the recursor prefix checks the

motive’s target universe and elimination restrictions. A higher-level candidate service should therefore perform four operations: construct a dependency-closed generalization recipe; preprocess compound indices where necessary; invoke the native induction machinery transactionally; and inspect the resulting typed subgoals and substitutions before committing.

Classify failures rather than swallowing them all as “no solution.” Useful categories include an unavailable or malformed recursor, compound-index preprocessing required, a missing instance, an illegal elimination from a proposition into data, an ill-typed motive, unresolved unification, and resource exhaustion. Interrupts must propagate. The API does not by itself discover which extra accumulator invariant to invent, but it can validate many cheap alternatives and give actionable diagnostics.

Return a replay recipe or a snapshot-scoped result. Raw subgoal identifiers from a rolled-back context must not escape into another branch. Test the service on vector map, vector zip, dependent pairs, repeated indices, and definitions whose target family depends on the major value itself. Keep tests pinned to the toolchain because metaprogramming interfaces can change. Canonical’s existing support for indexed inductive reasoning is also an important baseline: the research question is how to integrate effective motive search into Leant2’s architecture, not whether such reasoning is possible in principle.[12]

7 R5: General recursion with explicit obligations

7.1 R5.Q1. A capability interpreter and Lean’s recursors

Question, paraphrased. Will a synthesis-time interpreter agree definitionally with `brecOn` or `WellFounded.fix`, and how should evaluation and publication work?

Answer. Not automatically. Fix a domain D , a family $B : D \rightarrow \text{Sort } u$, a relation R on D , and a proof that R is well founded. A recursive body should have the shape

$$F : \prod_{x:D} \left(\prod_{y:D} R(y, x) \rightarrow B(y) \right) \rightarrow B(x). \quad (5)$$

At the current input x , the inner function is a capability to call recursively only at a smaller y , with an explicit decrease proof. Its dependent result is $B(y)$, not always $B(x)$ or a fixed output type. Synthesis may leave the decrease proof as an obligation; it must not give the body an unrestricted self-reference and hope the final gate discovers termination later.

The well-founded fixpoint f satisfies the equation

$$f(x) = F(x, \lambda y \ h. f(y)).$$

Whether a particular equation is definitional depends on the representation, recursor, proof terms, and reduction policy. Lean’s general well-founded equations are normally reasoned about propositionally. There is also a version-sensitive exception worth knowing: Lean 4.27 introduced a reducible fuel-based implementation for certain natural-number-measure definitions. One should neither promise universal definitional agreement nor repeat the outdated blanket claim that no well-founded definition can reduce in the kernel.[8, 9]

An independent synthesis interpreter needs a simulation theorem. For a fuel interpreter, a useful statement is: if evaluation at x returns v with enough fuel, then $v = f(x)$; and, when a finite recursion-height bound is available, evaluation above that bound succeeds. Prove it by induction on the evaluation derivation or by well-founded induction on R . Fuel exhaustion proves nothing about the contract. A type-correct interpreter is not, merely by having the same input/output types, an implementation of the same function.

A practical publication path already suggested by Lean itself

The 4.34.0 standard library contains a directly relevant precedent. It defines `Nat.recCompiled`, proves `Nat.rec_eq_recCompiled`, and marks the equality with `@[csimp]`. Its comment explicitly identifies the compiler’s recursor-support limitation as the reason. This is a proof-backed compilation rewrite, not an unproved `implemented_by` substitution.[7]

Leant2 should investigate this route for its recursor publication problem before developing a universal custom interpreter. Synthesize the semantic recursor term; construct or reuse an ordinary recursive definition; prove their equality, or directly reprove the contract for the published definition; then use an appropriate proof-backed compilation rewrite or publish the checked ordinary definition itself. An equality proof may use axioms such as those behind function extensionality, so the chosen policy must be audited rather than assumed unchanged. A pointwise or direct contract proof can sometimes avoid an unnecessarily strong equality obligation.

This still separates logic from runtime. A proof that two Lean definitions are equal certifies their logical relationship. Executable behavior additionally relies on the compiler/runtime and the absence of unaccounted-for implementation substitutions. Report successful export and compilation separately, and run representative observations on the exported artifact. Do not label a kernel-accepted but noncompiling recursor expression as an executable synthesized program.

7.2 R5.Q2. What top-down angelic recursion must preserve

Question, paraphrased. Can capabilities and angelic evaluation be combined with top-down synthesis without losing the useful composition properties of Burst?

Answer. They can, but the missing ingredient is a shared constraint representation, not a different spelling of recursion. A partial recursive-equation IR should record its formal parameters, guards, branches, recursive call sites, holes, and pending termination proofs. Every occurrence of a hole points to the same unknown function body. Different example executions constrain that body in different environments.

An angelic return value summarizes an assumption on a call; it is not an implementation. When the submitted examples specify a smaller call, its observed result can constrain the oracle. Otherwise use a symbolic value and accumulate relations imposed by the enclosing context. At realization time, the same synthesized body must satisfy all these relations and every decrease obligation. A local combination that works with mutually inconsistent recursive answers must be rejected or refined before publication.

The main advantage to retain from a bottom-up system is representation of many candidate subprograms through a shared summary. A purely depth-first top-down loop can repeatedly rediscover the same realizations and lose that sharing. Add a typed library of closed candidates and compact version spaces for a supported first-order fragment. Conversely, top-down hole contexts retain the local type and induction information that an excessively erased bottom-up representation would lose. The hybrid should exchange typed recipes and constraints rather than mutually incompatible internal languages.

Smyth is relevant here precisely because it addresses interdependent goals without trace-complete examples. Burst offers an angelic strengthening perspective. The 2026 Cataclyst work provides a further comparison point for recursive sketches and mixed-quantifier first-order properties, with counterexample-derived syntactic pruning; it is not an off-the-shelf dependent Lean solution.[14, 15, 28] The Leant2 evaluation should compare these architectural ideas on matched specification regimes instead of transferring their aggregate success rates to a different

language and workload.

7.3 R5.Q3. A small recursion basis and evidence for coverage

Question, paraphrased. Is there a canonical small collection of recursion schemes that covers the intended tasks, and what coverage can be claimed?

Answer. There is no unique useful “canonical basis” independent of the language and cost model. A small implementation basis is still sensible: structural recursion over one selected inductive argument with generalized parameters; product/function accumulators; and well-founded recursion through equation (5). Helper introduction should initially be bounded to one explicitly synthesized helper, with mutually recursive and more elaborate schemes introduced in later size grades.

The structural tier is more capable than it may appear. List zip can recurse on the first list while the motive retains the second as a parameter, performing a case analysis on it in the step. Indexed zip requires the corresponding dependent motive. A tree fold can combine the results of both recursive children. Fibonacci can use a pair carrier instead of unrestricted course-of-values recursion. These examples justify the proposed ordering, not a measured coverage percentage.

The well-founded tier handles tasks whose direct equations do not follow one structural subterm. Merge can use the sum of the two list lengths. Euclidean gcd can use the second natural argument, with the nonzero branch proving that the remainder is smaller. Filtering recursive arguments in quicksort requires a length argument and appropriate inequalities. These are not supplied by an arbitrary call capability for free: guard facts, arithmetic lemmas, and contract proof search must solve them.

Coverage should be measured by ablation on three complementary groups: elementary list/natural/tree exercises; the unresolved Church and indexed cases already in the repository; and adversarial variants that alter index expressions, parameter order, or termination measures without changing the essential algorithm. Count separately tasks expressible in the grammar, tasks for which a witness was found, tasks with a checked contract, and tasks successfully exported and executed. Give both logical-work and wall-clock curves. No experiment in this review establishes a coverage percentage for the proposed basis.

A useful diagnostic comes from a reference solution withheld from synthesis. Check whether that solution can be represented within each tier’s grammar and bound. If it cannot, a search failure is a coverage limitation. If it can, the failure is attributable to search order, checking cost, or implementation. Keeping these cases separate prevents a “faster search” claim from hiding the removal of difficult but intended programs.

8 R6: Contracts, inductive proofs, and proof debt

8.1 R6.Q1. Interleaving programs and proofs

Question, paraphrased. What discipline avoids both premature proof attempts and discovering too late that a chosen value cannot satisfy its contract? Is there a Lean-native analogue of refinement abstraction?

Answer. Use a dependency-driven scheduler over both value and proof obligations, with cheap propagation before expensive proof search. The proposal’s assertion that proving a property of a program with holes is not meaningful until those holes are filled is too strong. Reflexivity can prove $?n = ?n$; an equality proof obligation can constrain a witness; and a theorem quantified over the remaining holes can certify an entire family of completions. What is dangerous is

treating a proof about one temporary assignment as valid for all later assignments.

There are three legitimate ways a proof obligation can make progress. It may be solved uniformly for all unresolved dependencies. It may assign some of those dependencies, committing those choices within the current branch. Or it may produce a conditional lemma whose remaining assumptions become explicit obligations. A proof scheduler should record which of these happened. A tactic that merely times out has supplied none of them.

For a pointwise postcondition $Q(x, y)$, search for

$$g : \prod_x \{y : B(x) \mid Q(x, y)\}, \quad f(x) := (g(x)).\text{val}.$$

An inductive branch receives both the recursive result and its property, making many obligations local. This is a useful form of correct-by-construction synthesis. It does not automatically decompose arbitrary relational contracts. An involution law $f(f(x)) = x$, for example, couples different executions of f ; a homomorphism law couples several inputs. Such contracts require a stronger invariant, a suitable algebraic construction, or a global proof.

Represent an obligation by its dependency set, a readiness class, previous proof attempts, and residual facts. Run inexpensive definitional equality, reflexivity, constructor reasoning, and already applicable lemmas early. Run a more expensive arithmetic or induction tier when relevant value structure stabilizes or when the obligation is sufficiently constraining to justify its cost. The policy must allow exceptions to a rigid “all values first” order: a proof about an index may be exactly what determines a value hole.

A finite predicate abstraction can be implemented without an external SMT solver. Choose a finite, explicit vocabulary of predicates on the current values, and use Lean-proved implication and transfer lemmas to propagate abstract facts. For example, summaries may record length equalities, nonemptiness, or interval bounds. A failed implication attempt means *unknown*, not that the implication is false. Joins must forget unsupported facts rather than invent them. This is an engineering adaptation of refinement-style reasoning; it is not a complete abstraction of arbitrary Lean propositions. Synquid supplies a useful comparison point, but its solver-supported refinement fragment should not be conflated with unrestricted dependent proof search.[26]

8.2 R6.Q2. A recognizable class of automatic inductive contracts

Question, paraphrased. Which properties are reliably handled by induction along a synthesized program, simplification, and arithmetic? Can the engine identify that class?

Answer. Specify a sufficient, certificate-producing class rather than claiming to recognize every property that happens to admit such a proof. For a list fold with seed $z : K$, step $s : A \rightarrow K \rightarrow K$, and an invariant $I : \text{List } A \rightarrow K \rightarrow \text{Prop}$, require local certificates

$$I([], z), \tag{6}$$

$$\forall a \, xs \, k, I(xs, k) \rightarrow I(a :: xs, s(a, k)). \tag{7}$$

These imply the desired global fold invariant.

Proposition 8.1 (Local algebra certificates). *If equations (6) and (7) hold, then*

$$\forall xs, I(xs, \text{foldr } s \, z \, xs).$$

If additionally $I(xs, k) \rightarrow Q(xs, o(k))$, then the program $o \circ \text{foldr } s \, z$ satisfies $\forall xs, Q(xs, f(xs))$.

Proof. Induct on xs . The empty case is equation (6). In the constructor case, the induction hypothesis establishes $I(xs, \text{foldr } s \ z \ xs)$; applying equation (7) gives the required invariant for $a :: xs$. Apply the finishing implication to obtain the postcondition. $\square$

This pattern extends to polynomial inductive datatypes by giving one preservation obligation per constructor and one invariant hypothesis per recursive field. For indexed families, the predicate and recursive results must be typed at their respective indices. The extension is structural, but the generated obligations can still be difficult.

A recognizable *automation fragment* consists of annotated fold/recursor shapes, an invariant chosen from a fixed grammar, and local obligations which, after a terminating certified normalization, belong to a specified decidable theory. A useful initial theory is quantifier-free linear arithmetic over naturals and integers together with a fixed collection of constructor and projection facts. Universal branch variables become arbitrary local parameters; the resulting implication is discharged by the selected decision procedure. Completeness is then relative to the invariant grammar, normalization rules, and decision theory. It is not completeness for finding every inductive invariant.

For map-length preservation, choose $I(xs, ys)$ to be $\text{length}(ys) = \text{length}(xs)$. The empty case is immediate, and the cons case reduces to the recursive equality with one added to each side. An accumulator reversal needs a strengthened invariant involving the extra parameter; a property about sortedness or permutation needs corresponding logical structure and lemmas, not merely linear arithmetic. A syntactic recognizer should distinguish “all local obligations lie in our certified fragment” from “the invariant is plausible but a general prover is needed.”

This also clarifies the role of helper invention. Inventing a pair carrier without an invariant can make the program search easier while making certification harder. Synthesize the carrier, its local invariant, and the finishing implication as related but separately budgeted objects. Do not charge a proof failure against the semantic correctness of the program unless a checked counterexample is obtained.

8.3 R6.Q3. Available induction automation in Lean

Question, paraphrased. Which current Lean facilities can be used as bounded proof services, and what remains to be built?

Answer. The dependable core is an explicit induction/case-analysis adapter followed by ordinary proof-producing automation, not an assumption that a general-purpose tactic will invent the induction. Lean exposes the induction API discussed in R4; its tactic ecosystem provides simplification, arithmetic automation, and grind. Aesop’s own documentation explicitly says that induction is deliberately left to the user in its standard approach. Canonical demonstrates relevant dependent inhabitation and recursor search, but that is not a promise that unrestricted program invariants will be discovered automatically.[6, 5, 10, 11, 12]

For Leant2, implement a small proof service which reads the synthesized program’s recursion metadata, chooses its major argument, reverts the recorded generalized parameters, performs induction, unfolds the corresponding equation, and passes the resulting local goals to a tiered portfolio. The same metadata used to build the program should guide the proof; reconstructing it from a large opaque recursor expression is avoidable work. For a well-founded definition, use its checked equation and induction principle rather than pretending the body is definitionally structural.

LeanHammer is a relevant optional integration for premise selection and proof reconstruction; the premise-selection paper has an ICLR 2026 version, so the proposal’s preliminary bibliography should be updated. It does not remove the need to identify the induction or to validate all

reconstructed dependencies against the requested axiom policy. Its external infrastructure also makes it a different deployment profile from a Lean-core-only default.[27]

Rippling and lemma discovery suggest useful search heuristics, but this investigation did not establish a maintained, drop-in Lean 4.34 library that supplies the entire proposed automatic-induction service. The practical answer is therefore a small native adapter plus explicitly optional integrations. A proof service should return a closed proof, a checked counterexample, a suspended obligation, or an explicit resource failure. It should never collapse these outcomes into a single Boolean.

Budgeting must cover the actual expensive operation. A deadline checked only between tactic calls cannot preempt a single long call. Use supported heartbeat/interrupt mechanisms inside the worker, and distinguish soft cooperative cancellation from a hard process boundary where one is required. Preserve the frozen environment and reconstruct the proof in the acceptance process. Millisecond tiers may be useful measured defaults on one machine; they are not reproducible semantic bounds.

8.4 R6.Q4. Scheduling proof debt

Question, paraphrased. Is there a principled policy for spending time on proof obligations rather than more program search?

Answer. There is a principled engineering formulation, but no justified claim of a universally optimal bandit policy for this workload. Candidate branches share subterms, learned facts, and proof obligations; costs and success probabilities change as other goals are solved. This violates the simple independence and stationarity assumptions behind many elementary bandit models.

Start with a deterministic, fair policy. Cheap propagation runs at each relevant change. Expensive proof attempts receive increasing logical-work quanta, with separate queues for local branch certificates, termination proofs, and global postconditions. Reserve a nonzero share for older obligations so that promising new candidates cannot starve a correct existing one. Reuse a proof only when its dependency key still matches. A proof solved once for a generalized helper can be more valuable than several isolated global proof attempts.

An optional priority estimate is

$$\frac{\text{Pr}(\text{certification or useful refutation} \mid \text{features}) \cdot \text{expected downstream work saved}}{\text{estimated additional work} + \text{switching overhead}}.$$

This is a scheduling heuristic, not a theorem about optimality. Learn it only after logging enough matched obligations to estimate both successes and unsuccessful costs. Freeze or record the estimator for replay; keep the certificate-producing services independent of it.

Checked counterexamples should be shared across candidates for the same frozen contract, including all necessary preconditions on their inputs. Survivors are tested on those counterexamples before expensive proofs. By contrast, “the prover failed on this program” must not become a shared negative example. The distinction preserves the rule that tests can refute a universal claim at a concrete point, while only a proof certifies the universal claim.

9 R7: Retrieval without forgetting the dependencies

9.1 R7.Q1. A useful dependent type abstraction

Question, paraphrased. How can abstract reachability over universe-polymorphic, dependent signatures remain a sound over-approximation without connecting everything to everything?

Answer. Use a hierarchy of relational type abstractions and an AND-hypergraph of component applications. A declaration with several arguments is not an ordinary edge that becomes usable when just one argument type is reachable. Its application requires all arguments, with shared type parameters and index constraints instantiated consistently. The abstraction can relax these conditions, but must not silently turn a failed approximate match into a proof of concrete impossibility.

At the coarsest useful level, retain the conclusion head, argument heads, binder roles, and whether the result is a proposition, a data type, or a type former. At the next level retain repeated type variables, parameter positions, and a small set of index relationships. For example,

$$\text{append} : \text{Vec } A \ n \rightarrow \text{Vec } A \ m \rightarrow \text{Vec } A \ (n + m)$$

shares the same A across all three vector occurrences, but does not equate n and m . Forgetting the arithmetic equation is a legitimate over-approximation; introducing an incorrect equality between the indices is not. Universes should be represented by compatible symbolic constraints or an explicit unknown, never silently specialized to a convenient universe.

A concrete typing derivation must map to an abstract derivation. This is the central soundness obligation of the abstraction. It licenses rejecting an abstractly unreachable expression class only when the abstraction and reachability computation really cover every concrete construction in the claimed grammar. Lambda introduction, local variables, constructors, allowed casts, and all admitted component forms must be included in that coverage. A graph of library constants alone does not cover a language that can synthesize higher-order arguments.

The concrete stage instantiates the declaration's full telescope in a fresh transactional context and checks the resulting arguments and conclusion. Failed unification can identify a spurious abstract connection, but only a suitably justified failure supports a reusable abstraction refinement. A timeout or unsupported elaboration case is not such a justification. Repeatedly rejecting individual concrete candidates, without changing the abstraction, is ordinary concrete filtering rather than type-guided abstraction refinement.

Start with a fast index and a relevance ranking, not a hard global exclusion filter. Fixed top- k retrieval and a path bound of three can be useful speed controls, but change the searched grammar. Include them in the result's bound descriptor. A later fair expansion of the provider set can preserve an enumerative fallback. Output-sensitive behavior matters: no discrimination tree can return a million matches in time independent of the number returned.[2, 17]

9.2 R7.Q2. Type classes have already been addressed in TYGAR

Question, paraphrased. Has type-guided abstraction refinement been combined with type classes, and what should Lean preserve?

Answer. Yes. This is a concrete answer missing from the proposal: Hoogle+ explicitly handles type classes by dictionary passing. Class methods consume dictionary components, while instances supply components that construct dictionaries. Its paper discusses this in its type-class treatment and implementation. The formal setting and supported Haskell subset are nevertheless narrower than dependent Lean; higher-kinded polymorphism is not generally

covered. The reported evaluation uses 291 components and 44 queries, so it should not be cited as a demonstrated experiment on several hundred thousand dependent declarations.[17]

The corresponding Lean adaptation should keep instance arguments as typed obligations within the same dependency problem as ordinary arguments. An instance can determine an output type or depend on an unresolved index. Conversely, an ordinary argument may determine the type needed for instance synthesis. Searching instances eagerly for every approximate component match can be expensive; concluding “no instance exists” before the relevant metavariables are fixed can be wrong.

Cache instance results under the full class application, universe information, local-instance context, environment fingerprint, and relevant options. Distinguish an established result from a postponed search and from failure under a particular resource bound. Class parameters cannot be reduced to a single (class, type) pair in general. In particular, output-parameter behavior and multiple class arguments carry information that must remain visible to the elaborator.

A class dictionary is not necessarily a proof-irrelevant object: it may contain operations that affect the program’s behavior. Two dictionaries for the same carrier type can implement different comparisons. Observational evaluation and memoization must therefore retain the actual instance identity or a proved equivalence sufficient for the intended use. Class laws should be available as explicit evidence when a pruning rule depends on them.

Finally, ordinary Lean permits weakening and contraction. A component argument need not consume the only available value of its type, and one dictionary can be reused. A Petri-net-inspired representation must encode this ordinary functional behavior rather than accidentally imposing linear-resource discipline. The abstract hypergraph should express possible composition; exact dependent type checking determines whether a particular composition is realizable.

9.3 R7.Q3. Premise selection for data programs

Question, paraphrased. Can a premise selector accept a data goal with examples, and can a training corpus be obtained by erasing library definition bodies?

Answer. A wrapper can submit a data goal or an encoding of its signature to an existing relevance model, but that does not establish that the model understands the examples or ranks executable components well. Proof premise selection is trained and evaluated on proof dependencies; program synthesis needs behavior-sensitive selection of functions, constructors, and helpers. LeanHammer is a useful interface and evaluation comparison, not evidence that this transfer already works for Leant2.[27]

Define a retrieval query containing the exact target type, local telescope, symbols appearing in the contract, and a typed summary of the observations. For lists this can include input/output length relationships, preserved elements, ordering patterns, and whether examples distinguish empty and nonempty cases. These summaries are heuristic features unless justified by a checked theorem. The retrieval reply should be a bounded ordered list of declaration names and optional instantiation sketches, never an accepted proof or a reason to discard all other providers.

A Lean-native baseline should combine a discrimination-tree lookup, symbol overlap, type-shape matching, and a small demand-directed composition search. It needs no external model or server. Compare learned retrieval against this baseline at equal downstream synthesis budgets, rather than comparing retrieval scores alone. The important metric is additional fully certified programs per unit of total work, including the cost of loading the index and evaluating the proposed components.

Erased-definition tasks can supply supervision, but careless construction makes them trivial. Hiding the body text while leaving the original declaration callable allows direct retrieval of the answer. Aliases, generated equation lemmas, implementation rewrites, and closely related

definitions can also reveal it. Construct a controlled provider environment that excludes the target and designated revealing dependencies, document the removal policy, and keep a separate unrestricted-library track for realistic reuse. A theorem about a hidden target must not silently reintroduce that target as a callable constant.

Split training and evaluation by definition family and preferably by module or repository, not merely by randomly choosing individual declarations. Near-duplicate variants of the same algorithm should stay in one split. Record which reference-body constants provided supervision, but do not expose those labels at inference time. Likewise, example generation must be separate from an inaccessible reference implementation during synthesis. Universal specification lemmas can serve as contracts only when their statements remain well defined in the controlled environment.

The natural corpus has at least two tracks: construction from a restricted primitive library, which measures synthesis, and reuse from a broad library, which measures retrieval and composition. A third, indexed-family track tests whether type abstraction actually preserves the constraints that matter. No accuracy number for these tracks can be inferred from a proof-premise benchmark; they need direct experiments.

10 R8: Ranking and equivalence are different questions

10.1 R8.Q1. A defensible ranking objective

Question, paraphrased. Can ranking inhabitants be given a formal objective such as minimum description length, and should it agree with user preference?

Answer. Yes to a formal objective, no to an automatic identification of that objective with a user's intent. Treat typing, the exact contract, and the axiom policy as hard constraints. Within that feasible set, one possible score is

$$J(f) = L_G(f) + \lambda_H L(\text{new helpers}) + \lambda_R \widehat{\text{runtime}}(f) + \lambda_U U(f),$$

where L_G is a description length under a fixed grammar, the helper term charges definitions not already in the environment, runtime is a declared estimate or measured observation, and U is a soft penalty for suspiciously unused information. All weights, normalization choices, and dictionary costs belong to the ranking model's version. An alternative is a lexicographic score that avoids pretending the units are naturally interchangeable.

If a probabilistic grammar is used, $L_G(f) = -\log P_G(f)$ gives the usual coding interpretation, subject to a properly specified probability model. Every legal production must have a usable finite grade in the completeness lane, and zero-cost cycles must not prevent progress. A learned prior can affect preference without affecting the truth of the contract, as in cost-guided synthesis, but its training distribution is not a theorem about the current user's intended program.[16]

The current syntactic unused-input heuristic deserves particular caution. Projection functions legitimately ignore an argument. A contract may require a constant. Conversely, a term can mention a variable in a branch whose two results are identical without depending semantically on that variable. Syntactic use is an inexpensive feature, not a decision procedure for information flow. Penalize it softly, explain the feature, and let contract evidence override it.

Proof size should normally be a separate certification or presentation cost, rather than letting a long proof hide an excellent short program. Proof irrelevance does not mean the checker spends zero resources on every proof. Similarly, a raw recursor may be small as an expression while an exported ordinary definition is easier to read or execute. Report the semantic program, its proof, and its presentation separately, with an explicit relationship between them.

A cost-ordered search proves that a candidate is optimal only when its frontier lower bound uses this same fixed objective and all cheaper relevant alternatives have been accounted for. Adaptive priorities, top- k retrieval, a first-success grace period, and a changing grammar invalidate that simple conclusion. A truthful interface can say “best among candidates found” without saying “minimum.” If it proves bounded optimality, include the completed bound and cost-model fingerprint in the receipt.

Human preference should be evaluated directly. Record pairwise preferences between contract-consistent candidates, reference-solution rank, time to a useful answer, and whether diversity helps users disambiguate the task. The reference solution is not necessarily the unique correct answer, so rank metrics should distinguish exact reference syntax, proved semantic equivalence, and another valid implementation. No empirical user-preference result is established by this review.

10.2 R8.Q2. What to merge, and what merely to group

Question, paraphrased. For dependent programs, should equality mean definitional equality, equality after ignoring proofs, or agreement on generated inputs?

Answer. Keep three separate relations. *Definitional equality* is a kernel-level compatibility notion at aligned types. *Proved equality* includes suitable extensional equalities established by Lean. *Agreement on a finite probe set* is an observational signature used for display and ranking. The last is not a replacement for either of the first two.

For closed, same-typed candidates, bounded definitional-equality checks are a useful first deduplication stage. They should run transactionally: a meta-level equality check on terms with holes may assign metavariables, which would otherwise mutate search. Failure or a resource limit is not proof of inequality. Hashing normalized syntax accelerates a sufficient syntactic comparison; it does not make full conversion free or complete under a time bound.

For dependent types, the types themselves matter. Values at different indices cannot simply be compared after deleting all casts. First establish the type alignment or an appropriate dependent relation; then compare the values with that alignment recorded. Proof irrelevance identifies proofs of the same proposition in the relevant setting, not arbitrary data or transports between unrelated endpoint types. The typed cast discipline of R4 should therefore also govern ranking.

Finite observations are valuable for diversity. Group candidates by their outputs on the contract’s concrete inputs and additional deterministic probes, show a representative from each group, and retain the remaining members for later refinement. A distinguishing input is an explanation of a difference, not a proof that either candidate is wrong. For an empty or weak probe set, aggressive grouping can hide essentially every interesting behavior.

There is an elementary obstruction even for purely parametric list programs. Fix m . The identity and the function that returns its input up to length m but returns the empty list above that length agree on every input of length at most m and disagree at length $m + 1$. Both are element-parametric. Thus parametricity reduces the *element alphabet* needed for each length; it does not remove the need to consider unbounded lengths. The supplied finite checks instantiate this example at $m = 7$.

Observational pruning inside synthesis needs stronger conditions than grouping finished answers. The equivalence must be a congruence for all admitted enclosing contexts and all relevant evaluation environments, and representative replacement must preserve the cost or feasibility bound. Shared holes, recursively generated calls, dependent indices, and new counterexamples can invalidate a previously adequate finite signature. Keep alternative recipes or regenerate the bucket when observations change. Permanent deletion is warranted only with the corresponding fragment-specific theorem or certificate.

The implementation currently deduplicates candidates by pretty-printed text. That is a presentation key, not a semantic one: notation, implicit arguments, and proof elision can hide differences. Replace it by a typed structural identity layer, optional bounded conversion/proved-equality merging, and a separately labelled observational grouping layer.[4] This gives both reliability and useful variety without promising a cheap decision procedure for general program equivalence.

11 R9: Learned proposals with reproducible verification

11.1 R9.Q1. Accuracy of carrier and helper proposals

Question, paraphrased. What accuracy do current models achieve on the exact carrier-invention task, and how should the proposed experiment be designed?

Answer. The examined sources do not provide an accuracy estimate that can be transferred to this precise Leant2 setting: a frozen Lean type, a small contract, a specified carrier/helper grammar, a fixed realization budget, and a final Lean contract proof. This is a missing measurement, not evidence of low accuracy and not a reason to abandon the approach. General synthesis or proof-premise success rates would answer a different question.

Evaluate the proposal stage and the complete system separately. A suggested carrier can parse and type-check yet be useless within the available step/finisher grammar. Conversely, a mathematically sufficient carrier can fail because its realization opens an intractable subtree. The primary endpoint should be the number of fully certified programs at equal total work or equal wall-clock budget; intermediate endpoints explain why it changes.

Use held-out task families spanning option state, tupled state, continuations, indexed motives, generalized accumulators, and invented helpers. Remove reference carriers and helper definitions from prompts. Include structural variants that change parameter order, index expressions, or examples without merely changing variable names. Keep closely related variants within one train/evaluation split to limit contamination. Report textbook tasks separately from deliberately novel combinations.

Compare at least the native grammar, model-proposed carriers/helpers, direct model-proposed programs, and an oracle-carrier condition in which the reference carrier is supplied but its realization is still synthesized. The oracle condition distinguishes a weak proposal mechanism from a weak body synthesizer. At fixed proposal counts such as $k = 1, 3, 10$, record parsing/type validity, grammar membership, sufficient-carrier rate under a declared realization bound, checked-contract success, axiom compliance, and export/compilation success.

Include proposal latency, cold and warm index/model costs, downstream search work, and failed proposals in the totals. A high top- k suggestion rate can lose overall if requesting and elaborating the suggestions consumes the interactive budget. Use paired comparisons on the same tasks and confidence intervals that respect task-family grouping. Publish ordered proposal transcripts so that downstream search can be reproduced independently of model service availability.

No model was run for this review, and no proposal-accuracy percentage is asserted. The concrete recommendation is to begin with one bounded proposal batch outside the inner loop, targeting the carrier, motive-generalization set, or helper signature. This exposes the largest possible structural benefit while keeping the experiment interpretable. Learned premise selection, represented by the current LeanHammer work, is a useful separate baseline but should not substitute for measuring this task.[27]

11.2 R9.Q2. Determinism and auditability beyond receipts

Question, paraphrased. Can the model’s contribution be made deterministic enough for correctness-sensitive use, and what should be recorded?

Answer. Separate *verification reproducibility*, *search replay*, and *model regeneration*. A closed program and contract proof can be checked without the model. A search can replay an ordered proposal transcript without regenerating it. Reproducing the same transcript from a model requires additional control over weights, tokenizer, prompt format, inference implementation, numerical behavior, batching, and tie-breaking. A fixed seed or temperature zero alone does not establish all these conditions.

For a locally controlled model, record immutable model and tokenizer identifiers, quantization and inference settings, prompt-template version, canonical serialized query, context-selection policy, and ordered outputs. For a remote service whose implementation is not fixed by the client, promise transcript replay rather than bit-for-bit regeneration. In either case, the certificate should be independent of the model’s claimed confidence or explanation.

The safest proposal interface returns data: declaration names, carrier ASTs, bounded typed sketches, and explicit generalization choices. Do not execute arbitrary proposed commands, import directives, metaprograms, or tactic code in the authoritative environment. Validate the proposal grammar, node/depth limits, allowed constants, and scope before elaboration in a scratch context. Newly synthesized helper declarations should be dependency-checked and kernel-checked before becoming available to later steps.

The acceptance boundary must compare the result with the frozen query, not with a type or contract supplied by the proposer. It must also audit transitive axioms, reject escaped locals and unresolved holes, and preserve the declared execution policy. Resource limits remain necessary even for rejected proposals: a malicious or accidental oversized expression can consume excessive parsing, elaboration, or reduction resources without ever reaching the kernel gate. Optional remote prompting should have a separate, explicit policy for potentially private local definitions and contracts.

The proposal’s “only reorder, never change the set” rule requires an actual membership test. A carrier outside the finite native grammar is not a reordering. Either translate it into a recipe already within the fixed grammar or put it in a clearly labelled extension lane with grammar $\mathcal{G}^+$. A native fair fallback preserves eventual exploration under its stated assumptions, not identical finite-deadline results. An extension lane cannot justify an impossibility claim about the original or expanded language merely because it found nothing.

For auditability, a receipt should contain the original query and environment digest, search-grammar and rank-model versions, proposal transcript, exact universe treatment, work ledger, closed program, exact contract proof, transitive axiom inventory, and export relationship. Record unknowns, resource cuts, and the origin of each proposed alternative. A replay command should verify the certificate before optionally replaying search; correctness verification must not require repeating a statistical choice.

The default should remain Lean-native and model-free. An optional model can be an effective source of hypotheses, especially for carriers and helpers, but the same proposal interface should also serve deterministic grammars and user sketches. This makes model guidance replaceable and keeps the research question about useful proposals separate from the trust question about accepted results.

12 An integrated implementation plan

12.1 One native state, several explicit kinds of evidence

The recommendations do not require replacing Lean’s expression language with an unrelated theorem prover. Keep native Expr, native local contexts, and native transactional metavariable state as the authoritative substrate. The new layer is a search controller with explicit dependency and evidence records. Separate the following objects: a frozen query; a branch state; a constraint store; an enumerative recipe; a proof-service request; and an accepted, query-bound certificate.

A branch state contains a saved native state, open goals, the partial root program, accumulated fixed cost, and a deterministic recipe path. Its constraint store contains typed observations, dependencies on assigned as well as unassigned metavariables, oracle assumptions, termination obligations, and local invariant obligations. Its scheduling metadata contains estimated usefulness, proof debt, and logical-work charges. The fixed cost belongs to the grammar; estimated usefulness belongs to the scheduler. Neither is the same as elapsed time.

Use recipes to move proposals between branches and to persist search. A recipe names an allowed operation and its arguments; replay allocates fresh metavariables in the destination context. It must not consist of an Expr whose free-variable identifiers belong to a discarded branch. Revalidate imported recipes in the current frozen environment. Newly introduced helpers form an explicit acyclic dependency sequence of declarations, each checked before use; future mutual recursion needs a separate supported realization rule.

The essential controller can be expressed schematically as follows. This is architectural pseudocode, not a claim of compiled Lean code.

```
while logical work remains and no external cancellation:
  select a branch/proof obligation by the replayable scheduler
  restore the branch's native state
  propagate assignments and update dependency records
  apply only supported, completion-stable reductions
  if a checked exclusion certificate applies:
    prune this branch with the certificate's exact scope
  else if the program and required evidence are closed:
    check against the original frozen query and axiom policy
    record accepted certificate; attempt checked publication
  else:
    enumerate or replay the next permitted recipe
    elaborate it transactionally
    record children, dependencies, costs, and pending proofs
    charge work outside rollback and save resumable progress
```

The existing transaction module already places its work ledger outside rollback; retain that good design. The existing global residual-blocker memo, by contrast, should evolve into a dependency-aware per-observation mechanism before being used across a large persistent frontier. Snapshot cost must be measured again after this change: cheap snapshots along one DFS path do not imply cheap retention of thousands of frontier states and caches.[4]

12.2 Recommended implementation order

Phase 1: harden the observable contract. Bind acceptance to the frozen target and contract; make universe specialization a separate explicit outcome; apply the requested axiom policy to negative certificates; retain those certificates; and separate proof certification from executable publication. Add receipts and verification replay. Investigate proof-backed csimp publication

using the standard-library precedent. This phase fixes the meaning of success and failure before changing the search.

Phase 2: improve observations without replacing the evaluator. Decompose supported contracts through checked equivalences or necessary-condition lemmas. Keep true/false/s-tuck/resource states separately. Cache closed observations first; then introduce explicit read sets and assignment-aware invalidation. Use richer residual information for ranking and goal ordering before allowing any new custom-evaluator refutation rule. Differential tests should include open typed contexts, delayed substitutions, and rollback histories.

Phase 3: widen the useful native grammar. Enable a bounded outer structural recursion rule for naturals, dependency-closed indexed induction, and a small set of conditional/comparison components. Introduce product, option, and continuation carriers through one graded grammar, with a sketch path that supplies the carrier explicitly. Add local algebra invariants and an induction-along-the-program proof adapter. The paired “carrier given” and “carrier invented” tracks will show where the remaining difficulty lies.

Phase 4: expose a persistent frontier and principled retrieval. Refactor the continuation search into replayable whole states while retaining DFS as a policy for regression comparison. Add fixed-cost enumeration, fair scheduling, and conservative dependency components. Use abstract reachability as a ranking aid initially; promote it to a hard filter only with the required over-approximation argument and a stated grammar scope. Implement the small finite-carrier constraint solver as an exact bounded component, not as a universal carrier oracle.

Phase 5: add stronger semantic guidance. Develop the typed hole-closure fragment, backward observations, and capability-based recursive equations with checked realization. Add finite-domain angelic refinements where unsatisfiability can be justified, and keep unsupported cases as unknown. Introduce one-helper synthesis and invariant speculation with a separate cost grade. Upgrade proof debt scheduling using measurements rather than assuming a generic bandit policy is optimal.

Phase 6: evaluate learned proposals as an optional lane. Once frozen-query certificates, replay, and matched benchmarks exist, compare learned carrier/helper suggestions with the native grammar and the oracle-carrier condition. Record full costs and retain a model-free default. The core research result is a measured improvement in certified synthesis, not an impressive isolated suggestion.

This ordering differs from making the carrier problem a prerequisite for everything else. Its unbounded version is genuinely hard, but useful bounded carriers, sketches, and native motive support can already unblock other work. Conversely, safe pruning, result semantics, and publication should not wait for a successful invention algorithm.

12.3 Regression tests tied to the actual failure modes

A dedicated soundness-oriented suite should accompany ordinary functionality tests. The following table gives compact tests and what each protects. They are proposed implementation tests, not a claim that the current repository has been run against them.

Test family	Required behavior
Shared dependencies	Goals sharing type or universe metavariables, local let values, class arguments, or one relational observation must not be treated as independent merely because their displayed targets differ.
Rollback and cache	After a hole is assigned differently in sibling branches, a cached residual must be recomputed or validated against the full dependency key. Include an assignment–rollback–reassignment history.
Oracle consistency	Repeated occurrences of the same recursive call cannot choose inconsistent results; unknown calls must not be given a finite result domain and then globally refuted without a domain-completeness argument.
Boolean observations	A deliberately unusual BEq instance must preserve the exact Boolean contract. The checker may not silently replace it by propositional equality without the required law.
Dependent transport	An index mismatch must require the appropriate equality proof. Removing casts for display or ranking must not change the typed semantic term.
Proof/value interaction	A proof may uniformly solve an open expression or legitimately constrain a hole. A failed proof attempt must remain unknown, not become evidence that the value is wrong.
Frozen query	A valid term for a specialized universe or a weaker contract must not be returned as satisfying the original query.
Negative policy	A negative proof using an axiom outside the requested profile must not produce an accepted impossibility result; the actual retained certificate must replay.
Grammar scope	Exhaustion of one carrier, provider subset, motive class, or cost grade must not become impossibility of the target type or full contract.
Publication	A semantically accepted term which fails compilation remains certified but not executable. A replacement implementation needs a checked relationship and a separate runtime status.
External proposals	Escaped identifiers, undeclared dependencies, forbidden constants, oversized ASTs, and attempted environment changes must be rejected or safely bounded before authoritative acceptance.
Resource limits	Timeout, heartbeat exhaustion, unsupported evaluation, and incomplete class search must never be reported as a mathematical counterexample.

A particularly important adversarial test embeds two syntactically independent functions in one contract equation. It checks that the independence detector looks at semantic obligations, not only goal types. Another gives the same Boolean observations to many inequivalent polymorphic list programs; it checks that observational buckets can later split rather than permanently losing candidates.

12.4 A benchmark report that can support the claims

Preserve the published baseline as a regression target, but do not treat its saturated outcome counts as the only measure of progress. The proposal reports those measurements on a particular earlier commit and workstation; this review has not reproduced them.[3] Report the

existing Leant suites, unresolved stretch cases, indexed-family cases, general-recursion cases, and universal-contract cases separately. Distinguish closed-example certification from universal proof certification.

For each query, record grammar representability when a reference witness is available, time and logical work to the first candidate, time and work to the first certified candidate, export success, peak memory, provider candidates considered, failed unifications, observation reductions, and proof-service work. Report curves over completed work grades and wall-clock limits, not only one final solved count. Include cold versus warm imports and indexes, and disclose the compiler/runtime, operating system, processor, and parallelism.

Ablate one architectural change at a time. Useful comparisons include native kernel observations versus richer decomposition; no cache versus closed cache versus dependency cache; DFS versus fixed-cost frontier; fixed versus learned priority; supplied versus invented carrier; retrieval ranking versus hard bounded filtering; post-hoc proof versus local algebra certificates; and model-free versus transcript-guided synthesis. Measure memory along with time, because retaining alternate states and observational recipes is intentionally trading space for less repeated work.

Claims of determinism should use repeated runs with the same logical work, including perturbed worker speeds and different thread arrival orders. A wall-clock run may produce different prefix lengths and still obey the intended reproducibility contract. Claims of pruning completeness should use a bounded reference enumerator on small instances to verify that no valid in-bound witness disappears. Claims of positive correctness should replay every retained certificate, not merely inspect the number of successful searches.

12.5 What has and has not been resolved

The investigation supplies concrete answers where the question admits them: conservative independence conditions; safe lower bounds for coupled search; exact finite-domain carrier inference; continuation-based carrier lower bounds; a typed residual-certificate formulation; bounded angelic refinement conditions; an accessible Lean motive API; an existing proof-backed recursor compilation pattern; local invariant certificates; and a published dictionary-passing treatment of type classes in TYGAR.

It also identifies claims that need explicit hypotheses rather than a new heuristic: global parametricity of arbitrary Lean terms, negative conclusions from sample-limited carriers, observational replacement in dependent contexts, best-candidate guarantees under learned priorities, and equal results under equal wall-clock budgets. The mathematical arguments explain the missing hypotheses and propose operational replacements.

What remains empirical includes useful carrier-grammar size, the value of partial-success signals, memory cost of the frontier, effectiveness of invariant and helper proposals, data-goal retrieval accuracy, and model suggestion quality under the exact budget. The right next contribution is an implementation with the stated evidence boundaries and a reproducible ablation study. The article’s proposed architecture is intended to make those experiments interpretable, not to announce performance results that have not been measured.

A Question-by-question cross-reference

Every numbered item under “Questions for experts” in Proposal 11 is covered below. The source numbers abbreviate filenames under docs/proposals/11-further-improvements/sections/: 05 is 05-search.tex, 06 is 06-carriers.tex, 07 is 07-evaluation.tex, 08 is 08-dependent.tex, 09 is 09-recursion.tex, 10 is 10-contracts.tex, 11 is 11-retrieval.tex, and 12 is

12-ranking.tex. Source 12 contains both R8 and R9. Questions are paraphrased in the article rather than reproduced verbatim.[2]

Question	Source	Answer topic	Page
R1.Q1	05	Coupled goals, dependency components, and memoization	7
R1.Q2	05	Admissible lower bounds without double counting	8
R1.Q3	05	Adaptive search priority versus a fixed ranking objective	8
R1.Q4	05	Wall-clock limitations, deterministic prefixes, and replay	9
R2.Q1	06	Minimal carriers, finite constraint solving, and lower bounds	10
R2.Q2	06	Parametric functions, positions, and equality patterns	12
R2.Q3	06	Backward fusion, auxiliary summaries, and reversal	12
R2.Q4	06	Graded motive and carrier enumeration	13
R2.Q5	06	Generalization, failed proofs, and helper speculation	14
R3.Q1	07	Higher-order live evaluation and dependent hole closures	14
R3.Q2	07	Incremental reduction and rollback-safe invalidation	15
R3.Q3	07	Conservative evaluators and residual certificates	16
R3.Q4	07	Finite top-down angelic refinement and completeness	17
R4.Q1	08	Bounded motive classes and accumulator generalization	18
R4.Q2	08	Typed transports and limits of cast erasure	18
R4.Q3	08	Demand-driven unfolding and transparency	19
R4.Q4	08	Lean 4.34 motive-construction and induction APIs	19
R5.Q1	09	Recursive capabilities, simulation, and compilation	20
R5.Q2	09	Shared constraints in top-down angelic recursion	21
R5.Q3	09	A small recursion basis and honest coverage metrics	22
R6.Q1	10	Interleaving value and proof obligations	22
R6.Q2	10	Local algebra certificates and recognizable proof fragments	23
R6.Q3	10	Available Lean automation and a bounded proof adapter	24
R6.Q4	10	Fair proof-debt scheduling and its empirical model	25
R7.Q1	11	Relational abstract reachability for dependent signatures	26
R7.Q2	11	Hoogse+ dictionaries and Lean instance constraints	26
R7.Q3	11	Data-goal relevance and leakage-controlled corpora	27
R8.Q1	12	Description length, preferences, and bounded optimality	28
R8.Q2	12	Typed equivalence versus observational grouping	29
R9.Q1	12	A matched experiment for model-proposed carriers/helpers	30
R9.Q2	12	Proposal replay, model provenance, and query-bound checking	31

The nine rows in the proposal’s 15-experts.tex summarize these same areas; they are not nine additional independent question sets. The source question counts are $4+5+4+4+3+4+3+2+2 = 31$.

B Reproducible finite checks

The archive includes a dependency-free Python script, checks/check_design_examples.py, and its executed output, checks/results.json. These are checks of finite mathematical examples used in the design, not executions of Leant2 or evidence of synthesis speed. The script is deliberately small enough to inspect and rerun.

B.1 Carrier lower bound

Take alphabet $\{a, b\}$ and require a Boolean output indicating that both letters occur. The observation set contains every word of length at most three, hence $1 + 2 + 4 + 8 = 15$ words. The witness suffixes $[], [a], [b], [a, b]$ and left-prefix contexts $[], [a], [b]$ have respective observation rows

$$(0, 0, 0), \quad (0, 0, 1), \quad (0, 1, 0), \quad (1, 1, 1).$$

They are pairwise distinguishable. The carrier theorem therefore forces at least four reachable states for any deterministic right-fold realization with a final Boolean observation.

For a k -state carrier with initial state fixed to 0, the exhaustive script enumerates k^{2^k} step tables and 2^k output tables. Fixing the initial state loses no realizations up to relabelling. The tested table counts for $k = 1, 2, 3$ are respectively 2, 64, and 5,832, totaling 5,898. No table matches all fifteen observations. A four-state bit-pair carrier records whether each letter has occurred. The supplied implementation matches all 511 words of length at most eight.

The exhaustive search proves a finite negative statement about one-, two-, and three-state machines. The mathematical distinguishing-context argument gives the general lower bound; the bit-pair transition invariant gives the general four-state realization. Testing that realization on 511 words is a consistency check of the script, not a substitute for the invariant proof.

B.2 Other checks

The script compares reversal by a list-valued right fold with append, reversal by a continuation carrier, and ordinary sequence reversal on all 255 Boolean words of length at most seven. It checks the nontransitivity of compatibility between partial observation rows. It also constructs a rollback-cache counterexample, a toy shared-goal cost counterexample, and two polymorphic list behaviors agreeing through length seven but differing at length eight.

The goal-cost check uses an explicit abstract search model, not Lean’s unifier. The cache check demonstrates a dependency-key failure pattern, not a demonstrated bug trace from the repository. These distinctions are recorded in the JSON and should be retained when using the examples in tests or talks.

C Assumptions and certificate obligations

The strongest conclusions in the article depend on identifiable assumptions. A compact implementation checklist is useful:

Conclusion	Required evidence or assumption
Independent subsearch	Completion spaces factor; shared parameters are fixed; no omitted observation, type, universe, local-value, instance, or future-rule dependency crosses the partition.
Admissible heuristic	A proved relaxation lower bound, or disjoint cost accounting for independent components; fixed objective and sound handling of zero-cost transitions.
Carrier impossibility	A real finite-cardinality bound or a complete finite grammar search, with well-defined realization constraints; not a count of values discovered so far.
Canonical polymorphic tests	A suitable relational/naturality theorem for the admitted term fragment and dictionaries; input-length coverage stated separately.
Residual refutation	A substitution-stable checked exclusion certificate, or a proved simulation with conservative unknown cases and correctly scoped oracle assumptions.
Angelic completeness	Finite candidate/domain bounds where claimed, sound refinements, a total verifier in the finite theorem, and fair exploration.
Recursive execution	Well-foundedness/decrease evidence, a checked realization, and a simulation or sufficient equation correspondence; runtime trust reported separately.
Universal contract	Local invariant certificates plus the induction theorem, or a checked global proof; examples alone do not establish universality.
Retrieval exclusion	Every concrete derivation in the claimed grammar maps to the abstract search; resource failure is not abstract impossibility.
Candidate merging	Same-type definitional/proved equality or a fragment-specific contextual congruence; finite observations alone justify only grouping.
Negative result	Retained proof or a precise completed grammar/bound descriptor; original frozen target, universe treatment, and axiom policy preserved.
Replay	Pinned environment, query, grammar, policy, priority/model transcript, logical work, and stable tie-breaking; wall time is a separate measurement.

D Provenance and build instructions

The repository was inspected at commit 784664ff34e819422510a4d470ab07fe48bb736b; its toolchain file specifies `leanprover/lean4:v4.34.0`. The source proposal is dated 18 September 2026, and its performance evidence explicitly concerns commit 33cec8b. The article’s literature and documentation review date is 21 September 2026. Rolling documentation is identified as such; version-sensitive implementation recommendations use the inspected 4.34.0 source wherever available.

The selected implementation inspection covered the acceptance gate, result types, engine, transaction support, and relevant portions of the search core. It was a source review, not a

complete security audit or a build of the repository. No Lean compiler was available in the working environment. Consequently the proposed interfaces and snippets are architectural descriptions unless explicitly quoted as existing source, and no new Lean test pass or benchmark result is claimed.

To regenerate the article with a TeX distribution containing the listed packages, run `./build.sh` from the extracted archive, or invoke `pdflatex` on `leant2_expert_answers.tex` three times. The document uses local section files and an inline bibliography; there is no BibTeX step or downloaded figure dependency. Run `python3 checks/check_design_examples.py` to regenerate the finite-check results. The README distinguishes these commands and lists the included files.

References

- [1] Vladimir Reshetnikov. *Leant2 repository*, snapshot 784664ff34e819422510a4d470ab07fe48bb736b. Inspected 21 September 2026. Toolchain: Lean 4.34.0. <https://github.com/VladimirReshetnikov/Leant2/tree/784664ff34e819422510a4d470ab07fe48bb736b>.
- [2] Leant2 project. *Leant2: Design for Further Improvements*. 18 September 2026. Proposal 11, including the question sections 05–12 and the expert-question summary in section 15; at the snapshot in [1]. <https://github.com/VladimirReshetnikov/Leant2/tree/784664ff34e819422510a4d470ab07fe48bb736b/docs/proposals/11-further-improvements>.
- [3] Leant2 project. *Evidence*, section 01 of Proposal 11. Measurements reported for commit 33cec8b, not newly reproduced here. <https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/docs/proposals/11-further-improvements/sections/01-evidence.tex>.
- [4] Leant2 project. Implementation at the snapshot in [1]: `Leant2/Accept/Gate.lean`, `Leant2/Core/Types.lean`, `Leant2/Engine.lean`, `Leant2/Native/Transaction.lean`, and `Leant2/Search/Core.lean`. Selected source inspection, 21 September 2026. <https://github.com/VladimirReshetnikov/Leant2/tree/784664ff34e819422510a4d470ab07fe48bb736b/Leant2>.
- [5] Lean developers. *The Lean Language Reference*. Rolling reference consulted 21 September 2026; the displayed rolling version was 4.35.0-rc2. Version-specific recommendations in this article also use the pinned 4.34.0 sources below. <https://lean-lang.org/doc/reference/latest/>.
- [6] Lean developers. *Lean 4.34.0: Meta tactic induction implementation*. Contains `mkRecursorAppPrefix`, `getMajorTypeIndices`, and `MVarId.induction`. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Induction.lean>.
- [7] Lean developers. *Lean 4.34.0: Basic natural-number definitions and theorems*. Contains `Nat.recCompiled`, its `csimp` equality, and the arithmetic definitions discussed in the text. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Init/Data/Nat/Basic.lean>.
- [8] Lean developers. *Recursive Definitions*, in the Lean Language Reference. Consulted 21 September 2026. <https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/>.
- [9] Lean developers. *Lean 4.27.0 release notes*. 24 January 2026. See the natural-number-measure well-founded recursion change associated with pull request 7965. <https://lean-lang.org/doc/reference/latest/releases/v4.27.0/>.
- [10] Lean developers. *The grind tactic*, in the Lean Language Reference. Consulted 21 September 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [11] Jannis Limperg and Asta Halkjær From. *Aesop: White-Box Best-First Proof Search for Lean*. *CPP*, 2023. <https://doi.org/10.1145/3573105.3575671>. Also consulted the project’s README, particularly its discussion of induction and search state: <https://github.com/leanprover-community/aesop>.
- [12] Chase Norman and Jeremy Avigad. *Canonical for Automated Theorem Proving in Lean*. *ITP, LIPIcs* 352, article 14, 2025. Preprint: arXiv:2504.06239. <https://arxiv.org/abs/2504.06239>.
- [13] Chase Norman and Jeremy Avigad. *Implementing Dependent Type Theory Inhabitation and Unification*. arXiv:2603.01463, version 1, 2 March 2026. <https://arxiv.org/abs/2603.01463>.
- [14] Justin Lubin, Nick Collins, Cyrus Omar, and Ravi Chugh. *Program Sketching with Live Bidirectional Evaluation*. *Proceedings of the ACM on Programming Languages*, 4(ICFP), article 109, 2020.

- <https://doi.org/10.1145/3408991>. Preprint and full text: <https://arxiv.org/abs/1911.00583>.
- [15] Anders Miltner, Adrian Trejo Nuñez, Ana Brendel, Swarat Chaudhuri, and Isil Dillig. Bottom-up Synthesis of Recursive Functional Programs using Angelic Execution. *Proceedings of the ACM on Programming Languages*, 6(POPL), 2022. <https://doi.org/10.1145/3498682>. <https://arxiv.org/abs/2107.06253>.
- [16] Shraddha Barke, Hila Peleg, and Nadia Polikarpova. Just-in-Time Learning for Bottom-Up Enumerative Synthesis. *Proceedings of the ACM on Programming Languages*, 4(OOPSLA), 2020. <https://doi.org/10.1145/3428295>. <https://arxiv.org/abs/2010.08663>.
- [17] Zheng Guo, Michael James, David Justo, Jiaxiao Zhou, Ziteng Wang, Ranjit Jhala, and Nadia Polikarpova. Program Synthesis by Type-Guided Abstraction Refinement. *Proceedings of the ACM on Programming Languages*, 4(POPL), article 12, 2020. <https://doi.org/10.1145/3371080>. <https://arxiv.org/abs/1911.04091>.
- [18] Jeremy Gibbons, Graham Hutton, and Thorsten Altenkirch. When is a Function a Fold or an Unfold? *Electronic Notes in Theoretical Computer Science*, 44(1), 2001. Author-hosted bibliographic record: <https://www.cs.ox.ac.uk/publications/publication2368-abstract.html>.
- [19] Dana Angluin. Learning Regular Sets from Queries and Counterexamples. *Information and Computation*, 75(2):87–106, 1987. [https://doi.org/10.1016/0890-5401\(87\)90052-6](https://doi.org/10.1016/0890-5401(87)90052-6).
- [20] Jean-Philippe Bernardy, Patrik Jansson, and Koen Claessen. Testing Polymorphic Properties. *ESOP*, LNCS 6012, pp. 125–144, 2010. https://doi.org/10.1007/978-3-642-11957-6_8. Author-institution record: <https://research.chalmers.se/en/publication/99387>.
- [21] Moa Johansson, Dan Rosén, Nicholas Smallbone, and Koen Claessen. Hipster: Integrating Theory Exploration in a Proof Assistant. Preprint, arXiv:1405.3426, 2014. <https://arxiv.org/abs/1405.3426>.
- [22] Michael Luby, Alistair Sinclair, and David Zuckerman. Optimal Speedup of Las Vegas Algorithms. *Information Processing Letters*, 47:173–180, 1993. [https://doi.org/10.1016/0020-0190\(93\)90029-9](https://doi.org/10.1016/0020-0190(93)90029-9).
- [23] Austin Shen and Yunong Shi. Keep the Proof State Live: Snapshotting for Efficient Tactic Search in Lean 4. arXiv:2605.25556, version 1, 25 May 2026. <https://arxiv.org/abs/2605.25556>.
- [24] Mario Carneiro. Lean4Lean: Towards a Verified Typechecker for Lean, in Lean. Preprint, arXiv:2403.14064, 2024. <https://arxiv.org/abs/2403.14064>.
- [25] Łukasz Czapka. Practical Proof Search for Coq by Type Inhabitation. *IJCAR*, Part II, pp. 28–57, 2020. Open full text: <https://pmc.ncbi.nlm.nih.gov/articles/PMC7324019/>. Also consulted the CoqHammer documentation for sauto’s unfolding controls: <https://coqhammer.github.io/>.
- [26] Nadia Polikarpova, Ivan Kuraj, and Armando Solar-Lezama. Program Synthesis from Polymorphic Refinement Types. *PLDI*, 2016. <https://doi.org/10.1145/2908080.2908093>. <https://arxiv.org/abs/1510.08419>.
- [27] Thomas Zhu, Joshua Clune, Jeremy Avigad, Albert Q. Jiang, and Sean Welleck. Premise Selection for a Lean Hammer. *ICLR*, 2026. Preprint: arXiv:2506.07477, version 2, 25 February 2026. <https://arxiv.org/abs/2506.07477>. Conference record: <https://openreview.net/forum?id=m04JJNeRK6>.
- [28] Derek Egolf and Stavros Tripakis. Recursive Program Synthesis from Sketches and Mixed-Quantifier Properties. arXiv:2601.04045, version 2, 22 July 2026. <https://arxiv.org/abs/2601.04045>.